Algebraic and Computational Limits of LLM Guardrails

Joseph Robert Lopez

Abstract

LLM guardrails face four structurally distinct and independently provable barriers: algebraic blindness arising from syntactic monoid aperiodicity, information-theoretic loss formalized by the Fano bound, NP-hardness of instantiation verification, and structural transfer via functorial homomorphism combined with syntactic indistinguishability. Together these results characterize why inference-layer defenses are necessary but insufficient.

We operationalize these barriers through five attack vectors. V1–V4 (*homomorphic reasoning*: decomposition, zero-knowledge pipelines, Tree-of-Thought solving over abstract grammars, and encoding bootstrap) exploit the information-theoretic and computational barriers against abstraction-based attacks. V5 (*modular counting bypass*) exploits algebraic blindness: we prove that all substring-matching regex guardrails have aperiodic syntactic monoids and are therefore provably blind to any payload encoded using modular counting. Empirically, V3 yields a mean yield of 0.466 for BFS, 0.172 for random-beam, and 0.122 for LLM-guided Tree-of-Thought ($N{=}50$, seeds 0–49, $p{<}0.001$); BFS dominates, as exhaustive search over small synthetic grammars outperforms LLM heuristic pruning. We extracted syntactic monoids from a corpus of 123 patterns from eleven open-source guardrail tools and 19 additional author-generated test patterns (142 total); 100% are aperiodic, and the MOD_2 bypass construction succeeds against all aperiodic patterns. A 376-line proof-of-concept with three execution mediums validates all five vectors. We conclude that inference-layer guardrails are necessary but insufficient, and that effective defense must migrate to the execution layer where concrete artifacts become observable.

1 Prelude: The Guardrail Problem

This section requires no prior knowledge of formal language theory, circuit complexity, or abstract algebra. Every concept used in the technical sections is introduced here informally; the formal definitions follow in §5.

1.1 The Gate

A content filter reads a message and decides: safe or dangerous. The filter has a list of patterns. If the message matches any pattern, it is blocked. This is how every major AI deployment protects itself at the first layer—a pattern-matching gate between the user and the model.

Patterns are fast. A single pass over the input, linear time, runs before the model processes the request. For the threats they were designed for—known keywords, obvious phrasings, copy-pasted attack strings—they work. The question this paper asks is: what are they *structurally* unable to see?

1.2 Inside the Machine

A pattern like `bomb` compiles to a small machine that reads input one character at a time, following arrows between states:

$$\bullet \xrightarrow{b} \circ \xrightarrow{o} \circ \xrightarrow{m} \circ \xrightarrow{b} \bigstar$$

If the machine reaches $\bigstar$, the message is blocked. Otherwise it passes. This machine is a *deterministic finite automaton* (DFA). It has a fixed number of states and no external memory—it knows where it *is*, but not where it has *been*.

1.3 The Fingerprint

Every such machine has a mathematical fingerprint—a compact object that captures everything the machine can and cannot distinguish. Two messages that cause the machine to follow the same path are, from its perspective, *identical*. It cannot tell them apart. Not because it is poorly designed, but because the information needed to distinguish them does not exist within its state space.

This fingerprint is the *syntactic monoid* (§5.1)—the algebraic structure formed by collapsing all indistinguishable inputs.

1.4 The Lock That Never Unlocks

Once a keyword machine matches `bomb` and reaches $\bigstar$, it stays there forever:

$$\bigstar \xrightarrow{\text{any}} \bigstar \xrightarrow{\text{any}} \bigstar \xrightarrow{\text{any}} \bigstar \cdots$$

Applying the same operation twice produces the same result as applying it once. This property is called *aperiodicity*: the machine reaches a fixed point rather than cycling.

It is the algebraic signature of a machine that can match but cannot count.

A machine that *can* count—such as one recognizing strings with an even number of a's—cycles between states:

$$\text{EVEN} \xrightarrow{a} \text{ODD} \xrightarrow{a} \text{EVEN} \xrightarrow{a} \text{ODD} \cdots$$

This machine loops. Its fingerprint is *periodic*—it has a cycle, and it can track parity. Keyword filters have no such cycle. They are deadbolts, not light switches.

1.5 The Ceiling

Three classical theorems, proved between 1965 and 1992, chain together to establish the ceiling:

1. **Schützenberger (1965):** A machine with an aperiodic fingerprint recognizes a *star-free* language—one that can be defined using only AND, OR, NOT, and concatenation, without repetition counting.
2. **Barrington–Straubing–Thérien (1992):** Star-free languages correspond exactly to AC^0 circuits—constant-depth Boolean circuits with unbounded fan-in AND and OR gates.
3. **Furst–Saxe–Sipser (1981); Håstad (1987):** AC^0 circuits *cannot compute parity*—they cannot determine whether a count is even or odd, regardless of circuit size.

The chain: keyword pattern → aperiodic fingerprint → star-free language → AC^0 circuit → cannot count. Each link is a published theorem. The conclusion is not a conjecture; it is settled mathematics.

1.6 The Interleaving Attack

An attacker who knows the gate cannot count hides the payload using counting:

```
b   x   o   q   m   w   b   k
↑       ↑       ↑       ↑
b       o       m       b
```

The payload occupies odd positions; the filler is random. To decode, take every other character—a parity operation. The gate operates in AC^0. AC^0 cannot compute parity. The gate passes the message. It is algebraically blind.

This paper characterizes this blind spot precisely (§6), measures it across 142 production patterns (§8.1), identifies four additional attack vectors that exploit related structural barriers (§7), and constructs a defense that breaks the ceiling (§10.3).

2 Introduction

Production LLM deployments stack safety measures in depth: a regex pre-filter blocks known-harmful patterns before the request reaches the model, a neural classifier operating at inference time catches a broader distributional range of harmful intent, and an inference-time monitor inspects generated outputs before they are returned to the user. Each layer is individually sound for the threats it was designed to address. We show that each layer has a structural limitation provable from first principles, and that together these limitations leave a systematic gap an adversary can exploit without model access, fine-tuning, or gradient computation.

2.1 The Chinese Room Inversion

Searle's Chinese Room [30] argues that symbol manipulation without understanding is not intelligence. We observe that for the purposes of producing operationally correct procedures, it does not matter—the room produces correct answers regardless. An LLM operating as a theorem prover over an uninterpreted formal system produces derivations that, when instantiated via an external interpretation function held by the attacker, constitute valid operation sequences in real domains. The LLM never crosses the abstraction boundary. Understanding is unnecessary; structural fidelity is sufficient.

This inverts the usual framing: the Chinese Room is not a weakness of AI but a *weapon*. If an LLM can solve formal reasoning problems without understanding what they mean, it can solve harmful problems encoded as formal systems without triggering content-based guardrails—because the content visible to the guardrail is indistinguishable from legitimate formal reasoning.

2.2 Overview of Results and Paper Organization

We use *homomorphic* in its algebraic sense throughout: the interpretation functor $\hat{I}$ (Proposition 6.13) is a structure-preserving map from the free category of abstract derivations to the category of domain operations—a homomorphism of categories. The attack exploits this homomorphism: reasoning performed in the abstract category maps faithfully to operational sequences in the target domain.

Our contributions, organized by section, are:

1. **Algebraic impossibility** (sections 5 and 6): substring-matching regex guardrails have aperiodic syntactic monoids (Theorem 6.1) and are therefore provably blind to all modular counting encodings (Theorem 6.3; Corollary 6.4).
2. **Information-theoretic impossibility** (section 6.5): abstraction functions destroy safety-critical information;

the Fano bound (Proposition 6.9) gives an irreducible error floor for any lifted guardrail.

3. **Computational impossibility** (section 6.6): verifying whether an abstract program has a dangerous instantiation is NP-complete (Proposition 6.11).
4. **Structural impossibility** (sections 6.7 and 6.8): abstract derivations transfer faithfully to real domains via free-category functoriality (Proposition 6.13), and adversarial formal reasoning tasks are syntactically indistinguishable from legitimate ones (Proposition 6.15).
5. **Five attack vectors validated empirically** (section 7): V1–V4 demonstrate homomorphic reasoning; V5 demonstrates modular counting bypass.
6. **123-pattern regex corpus analysis plus validation** (section 8.1): confirms 100% aperiodicity across 123 patterns from eleven open-source guardrail tools plus 19 test patterns (142 total), with verified MOD_2 bypass constructions.
7. **Defense recommendations grounded in formal results** (section 10): each impossibility theorem maps to a specific layer of a defense-in-depth stack.

Scope. The algebraic blindness result (§5) applies specifically to regex guardrails and is unconditional. Transformer-based (neural) guardrails operate in TC^0 [8, 25], which *can* compute MOD_p, so the modular counting blind spot does not transfer. However, neural guardrails face the information-theoretic and computational barriers (§§6.5–6.6) and the structural indistinguishability result (§6.8), which apply at any layer that inspects only LLM-visible content.

3 Background and Related Work

3.1 Jailbreaking Taxonomies

Wei et al. [35] identify *mismatched generalization* as a failure mode in which safety training does not cover the full distribution of capability-eliciting inputs. Our abstraction bypass is a systematic instance: the model's formal reasoning capabilities generalize beyond the distribution on which safety training was performed. Zou et al. [42] demonstrate gradient-based adversarial suffixes; our attacks operate in semantic space rather than weight space and require no model access whatsoever. Xu et al. [38] provide an empirical taxonomy of guardrail bypass strategies spanning encoding, role-play, and indirect instruction attacks; our algebraic results explain why the encoding-based attacks in that taxonomy are provably undetectable by regex guardrails. The SoK by Li et al. [21] systematizes evaluation of jailbreak guardrails, highlighting the persistent gap between guardrail effectiveness and attacker capability—a gap our impossibility results formalize.

Esmradi et al. [12] introduce h4rm3l, a composable jailbreak language in which individual text-level transformations are composed to produce effective attacks. Our homomorphic reasoning framework generalizes composable string transformations: rather than composing text-level encodings, we compose structure-level abstractions whose compositional behavior is guaranteed by functoriality (Proposition 6.13).

Huang et al. [15] demonstrate that large reasoning models achieve a 97% attack success rate when deployed as autonomous jailbreak agents, confirming that LLM reasoning capability itself is the primary attack surface. This directly parallels our core thesis: the attack does not exploit model miscalibration but the model's genuine reasoning competence applied to abstractions the guardrail cannot interpret.

3.2 Encoding and Structural Attacks

StructuralSleight [34] achieves 94.62% attack success on GPT-4o via uncommon text-encoded structures (ASCII art, cipher text, etc.), demonstrating that encoding diversity defeats content filters. Our algebraic results explain *why*: any encoding that requires modular counting to decode lies outside the AC^0 closure of any aperiodic guardrail language (Theorem 6.3).

Boucher et al. [7] demonstrate imperceptible attacks via Unicode manipulation—inserting invisible characters between payload characters so that the guardrail sees an interrupted substring. Zero-width character insertion is an instance of MOD_2 interleaving with a null filler character; our Corollary 6.4 proves it cannot be patched within the regex paradigm.

The Content Concretization attack [2] traverses the abstraction hierarchy in the opposite direction: given an abstract harmful specification, the attacker prompts the LLM to supply concrete details. We proceed concrete-to-abstract—encoding a concrete harmful task as an abstract formal system—and the interpretation happens offline. The two approaches are complementary instances of the same general principle that abstraction boundaries defeat content inspection.

DictAttack [36] embeds malicious intent in grammar-level schema rules while maintaining benign surface prompts, achieving 94–99% attack success rates across multiple LLMs. This structured-output attack exploits the same abstraction boundary our V3 and V4 vectors formalize: the gap between surface-level content filtering and structural information encoding. Our contribution is the formal impossibility proof that regex guardrails *cannot* close this gap, whereas DictAttack provides empirical evidence of its exploitability.

S2C [4] is independent concurrent work (March 2026) that manipulates semantic reconstruction during inference

across multiple semantic dimensions, arriving at a complementary abstraction-based framing through a different research path. Where S2C operates within a single LLM inference pass, our pipeline distributes the harmful reasoning across multiple agents and an offline interpretation table; both works independently identify abstraction boundaries as the core attack surface.

3.3 Computational and Algebraic Impossibility

Ball et al. [5] prove that under standard cryptographic assumptions, adversarial prompts can be computationally indistinguishable from benign ones for any efficient filter. Our results are complementary. We establish *syntactic* (perfect, unconditional) indistinguishability for formal reasoning attacks (Proposition 6.15) and *algebraic* (unconditional) blindness for modular counting attacks (Theorem 6.3). Ball et al. give a more general but assumption-dependent result over a broader attack class; our results apply to specific attack constructions and carry no computational hardness assumption. Kumar et al. [20] propose certified defenses via randomized smoothing applied to LLM inputs, establishing provable safety radii around benign inputs under ℓ_p-bounded perturbations; our results identify a complementary regime where the attack is not a small perturbation but a structure-preserving map that lies outside the support of the smoothing distribution. Dong et al. [10] demonstrate a no-free-lunch tradeoff between guardrail thoroughness and computational cost, showing that any guardrail imposes overhead that scales with threat coverage; our results show that even unbounded overhead cannot patch the regex blindness established in Theorem 6.3. R2-Guard [39] replaces regex matching with reasoning-based evaluation via chain-of-thought; this approach operates in TC^0 and is not subject to the aperiodic blindness theorem, but remains susceptible to the information-theoretic and structural indistinguishability barriers established in §§6.5–6.8.

The abstract encoding channel in homomorphic reasoning is a form of *steganographic channel* [18]: the payload (harmful intent) is hidden in a covertext (legitimate formal reasoning task) such that no efficient observer (the guardrail) can distinguish stegotext from covertext. Hopper, Langford, and von Ahn [18] formalize perfect steganographic security as the requirement that no polynomial-time distinguisher achieves better than negligible advantage in separating channel transmissions from covertext. Proposition 6.15 can be viewed as establishing *perfect* steganographic security for the homomorphic reasoning channel: the distinguisher advantage is exactly zero (not merely negligible) because the LLM-visible content of the attack is syntactically equal to a legitimate reasoning task, making the distinguisher's task information-theoretically impossible rather than merely computationally hard.

3.4 Formal Languages and Neural Networks

Merrill [24] surveys the formal language expressivity of transformer architectures. Merrill and Sabharwal [25] establish that log-precision transformers are contained in TC^0, implying they can compute modular counting functions. Chiang et al. [8] give tighter expressivity bounds. Hahn [16] proves that self-attention mechanisms are limited on periodic formal languages, providing circuit-theoretic evidence for the same qualitative boundary our algebraic results formalize. Weiss et al. [37] extract DFAs from RNNs using the Angluin L^* algorithm [1]; the same DFA extraction pipeline underlies our empirical syntactic monoid computation.

3.5 Indirect Prompt Injection

Greshake et al. [14] establish that LLM-integrated applications blur the boundary between data and instructions, enabling an attacker to inject control flow via data channels. Our V2 (zero-knowledge pipeline) is a specific instance of this pattern: the "data" is a base64-encoded configuration that contains harmful parameters the agent processing it never reads. The theoretical underpinning differs—Greshake et al. exploit instruction/data confusion, while V2 exploits the information destruction formalized in Proposition 6.9—but the practical attack pattern is closely related.

3.6 Tool-Chain Composition Attacks

Zhou et al. [40] formalize the STAC (Simple Tool-Augmented Chain) attack class, in which individually benign tool calls compose into harmful pipelines—achieving $> 90\%$ attack success rate across 483 attack cases. Our V1 (decomposition) and V2 (zero-knowledge pipeline) are specific instances of this general pattern. The distinction is that STAC provides a large-scale empirical evaluation of tool-chain composition, while our contribution is the formal proof (Proposition 6.9, Proposition 6.11) that no inference-layer guardrail can detect such compositions without inspecting runtime state.

4 Threat Model and Formalization

4.1 Attacker and Defender Model

Attacker capabilities. The attacker has access to: (a) an LLM-based coding assistant with bash execution (e.g., Claude Code, Copilot, Cursor); (b) local LLM infrastructure (e.g., Ollama with open-weight models) for offline grammar solving without guardrail exposure; (c) standard

programming tools (Python, AST manipulation, base64 encoding).

Defender capabilities. The guardrail system operates at LLM inference time via RLHF training, system prompt rules, and output filters. It inspects the content visible to the LLM—prompts, generated code, tool calls—but not runtime arguments, environment variables, base64-encoded data in files the agent does not read, or offline interpretation tables.

Attacker goal. Cause the LLM to produce computation that, when instantiated with attacker-controlled parameters or interpreted via an attacker-held interpretation table, performs a task the guardrail would refuse if asked directly. The LLM must never observe the harmful intent in plaintext during any inference call subject to guardrail inspection.

Out of scope. Attacks requiring model weight access, fine-tuning, or gradient computation. Attacks against execution-layer defenses (sandboxing, network ACLs). Social engineering of human reviewers.

4.2 Abstraction-Based Formalization

Definition 4.1 (Abstraction function)**.** Let R be a finite set of concrete artifacts and M a finite set of abstract types. An *abstraction function* $U\colon R \to M$ maps each concrete artifact to its abstract type (e.g., $U(\texttt{streaming.example})$ = "video endpoint", $U(\mathrm{C_9H_8O_4})$ = "acetyl compound").

We do not require U to be surjective; we only use the existence of the fiber structure defined below.

Definition 4.2 (Fiber)**.** For $m \in M$, the *fiber* $U^{-1}(m) = \{r \in R : U(r) = m\}$ is the set of all concrete artifacts mapping to abstract type m. A fiber is *mixed* if it contains both allowed ($G(r) = 0$) and denied ($G(r) = 1$) artifacts under the ground-truth guardrail $G\colon R \to \{0, 1\}$.

Definition 4.3 (Lifted guardrail)**.** A *lifted guardrail* is a function $G'\colon M \to \{0, 1\}$ that approximates the ground-truth guardrail G through the abstraction layer: the system classifies concrete artifact r by evaluating $G'(U(r))$ rather than $G(r)$ directly.

The existence of mixed fibers is the root cause of the information-theoretic lower bound in Proposition 6.9: if a fiber $U^{-1}(m)$ contains both safe and unsafe concrete artifacts, no lifted guardrail G' can correctly classify all of them without access to r itself—but the guardrail operating at the abstraction layer sees only $m = U(r)$.

4.3 Guardrails as Language Recognizers

Let Σ be the finite alphabet of LLM prompt characters.

Definition 4.4 (Regex guardrail)**.** A *regex guardrail* is a function $G\colon \Sigma^* \to \{\mathsf{allow}, \mathsf{block}\}$ defined by a finite set of regular expression patterns $\{r_1, \ldots, r_k\}$ applied via substring search:

$$G(w) = \begin{cases} \mathsf{block} & \text{if } \exists\, i,\ \exists\, u, v \in \Sigma^* : w = u \cdot x \cdot v,\ x \in L(r_i), \\ \mathsf{allow} & \text{otherwise.} \end{cases}$$

Let $L_{\mathsf{safe}} = \{w \in \Sigma^* : G(w) = \mathsf{allow}\}$ be the set of strings the guardrail permits.

Definition 4.5 (Blocked language)**.** The *blocked language* of G is the complement $L_G = \Sigma^* \setminus L_{\mathsf{safe}} = \Sigma^* \cdot \left(\bigcup_{i=1}^{k} L(r_i)\right) \cdot \Sigma^*$. This is a regular language (closed under concatenation with Σ^* and finite union).

Definition 4.6 (Guardrail blindness to encoding class)**.** Let $\mathcal{E}$ be a class of invertible encodings $\phi\colon \Sigma^* \to \Sigma^*$. A guardrail G is *blind to encoding class* $\mathcal{E}$ if for every $\phi \in \mathcal{E}$:

1. $\phi(L_{\mathsf{blocked}}) \cap L_G = \emptyset$ (encoded harmful strings pass the guardrail), and
2. there is no regular language L' with aperiodic syntactic monoid such that $\phi(L_{\mathsf{blocked}}) \subseteq L' \subseteq \overline{L_{\mathsf{safe}}}$ (no aperiodic regex can separate encoded-harmful from genuinely-benign strings).

Condition (1) states that the encoding evades the current guardrail; condition (2) states that the evasion cannot be patched by adding any regex whose syntactic monoid is aperiodic—which by §5 is the entire aperiodic regex class. This pair of conditions connects the regex language-recognizer formalization to the abstraction-based formalization of §4.2: regex guardrails are a specific instance of lifted guardrails operating on string content, and their fibers are the equivalence classes of the syntactic congruence $\equiv_{L_G}$.

5 Algebraic Foundations

This section recalls the classical algebraic and circuit-theoretic machinery that underlies the impossibility results in §6. All results here are classical; we present them to establish notation and make the paper self-contained. Proofs and detailed developments can be found in Pin [28], Eilenberg [11], and Straubing [32].

5.1 Syntactic Monoids

Let Σ be a finite alphabet and $L \subseteq \Sigma^*$ a regular language. The *syntactic congruence* $\equiv_L$ is defined on Σ^* by

$$u \equiv_L v \iff \forall x, y \in \Sigma^* : (xuy \in L \Leftrightarrow xvy \in L).$$

Two strings are syntactically equivalent when they behave identically in every left-right context with respect to membership in L.

Definition 5.1 (Syntactic monoid)**.** The *syntactic monoid* of L is the quotient $M(L) = \Sigma^*/\equiv_L$, equipped with the monoid operation inherited from string concatenation.

$M(L)$ is finite by the Myhill–Nerode theorem [26, 27]: the number of $\equiv_L$-classes equals the number of distinct state-transformation functions on the minimal DFA for L. It is the smallest monoid recognizing L up to division [28].

Equivalently, $M(L)$ is isomorphic to the *transition monoid* of the minimal DFA for L: the monoid of all state-transformation functions $\delta_w \colon Q \to Q$ defined by $\delta_w(q) = \hat{\delta}(q, w)$ under function composition [11]. This equivalence makes $M(L)$ computable from the minimal DFA via standard monoid enumeration.

5.2 Aperiodicity and Star-Freeness

Definition 5.2 (Aperiodic monoid)**.** A monoid M is *aperiodic* (equivalently, *group-free*) if it contains no nontrivial subgroup, or equivalently, if every element has period 1: for all $m \in M$, there exists $n \geq 1$ such that $m^n = m^{n+1}$.

The prototypical non-aperiodic monoid is the cyclic group $(\mathbb{Z}/p\mathbb{Z}, +)$ for a prime $p \geq 2$: the generator $\bar{1}$ has additive order p, so $n \cdot \bar{1} \neq (n+1) \cdot \bar{1}$ for $0 \leq n < p-1$, and no power of $\bar{1}$ stabilizes before step p. This cyclic behavior is precisely what aperiodic monoids cannot replicate.

Theorem 5.3 (Schützenberger 1965, McNaughton–Papert 1971 [23, 29])**.** *For a regular language L, the following are equivalent:*

1. *$M(L)$ is aperiodic.*
2. *L is star-free: definable from Σ, $\emptyset$, $\{\varepsilon\}$, and single letters using Boolean operations and concatenation, without Kleene star.*
3. *L is definable in first-order logic with linear order,* $\mathrm{FO}[<]$.

Star-freeness and $\mathrm{FO}[<]$ definability are both "counting-free" in the sense that they can express positional and ordering constraints but cannot express modular periodicity. Theorem 5.3 makes this precise algebraically.

5.3 Circuit Complexity and Modular Counting

A language L is in AC^0 if it is recognized by a family of constant-depth, polynomial-size Boolean circuits with unbounded fan-in AND/OR gates. The class AC^0 is the circuit-complexity analogue of "polynomial time with limited depth," capturing exactly the computations that can be massively parallelized without counting.

Theorem 5.4 (Barrington–Compton–Straubing–Thérien 1992 [6])**.** *A regular language is in AC^0 if and only if its syntactic monoid is aperiodic.*

Theorem 5.5 (Furst–Saxe–Sipser 1981, Håstad 1987 [13, 17])**.** $\mathrm{PARITY} \notin \mathrm{AC}^0$. *More generally,* $\mathrm{MOD}_p \notin \mathrm{AC}^0$ *for any prime p, where* $\mathrm{MOD}_p = \{w \in \{0,1\}^* : |w|_1 \equiv 0 \pmod{p}\}$.

The chain Theorem 5.3 $\to$ Theorem 5.4 $\to$ Theorem 5.5 forms the backbone of the algebraic impossibility argument in §6: aperiodic monoid $\Rightarrow$ language in AC^0 $\Rightarrow$ language cannot compute MOD_p $\Rightarrow$ language is blind to any payload encoding requiring modular counting to decode.

5.4 Krohn–Rhodes Decomposition

Theorem 5.6 (Krohn–Rhodes 1965 [19])**.** *Every finite monoid M divides a wreath product of simple groups and aperiodic semigroups. The simple groups appearing in the decomposition (the* group components *of M) are uniquely determined up to isomorphism.*

The group components of $M(L)$ determine precisely which modular predicates L can express. Formally, by the variety-theoretic characterization of Eilenberg [11] and Straubing [32]: a language L can express the modular predicate MOD_p only if $M(L)$ contains a subgroup whose order is divisible by p, which requires $\mathbb{Z}/p\mathbb{Z}$ to divide some group component. The Krohn–Rhodes decomposition therefore gives a *complete spectrum of algebraic blind spots*: for any guardrail with syntactic monoid $M(L_G)$, the primes p for which G is potentially sensitive to MOD_p are precisely those dividing the order of some group component, and the guardrail is provably blind to all other primes.

When $M(L_G)$ is aperiodic (no group components), the guardrail is blind to MOD_p for every prime p—the blindness is total. This is the case for all substring-matching regex guardrails (Theorem 6.1, proved in §6).

6 Impossibility Results

6.1 Substring-Matching Patterns Are Aperiodic

Theorem 6.1 (Substring aperiodicity)**.** *Let r be a regular expression built from literals, character classes, bounded repetition, alternation, optional groups, dot, and Kleene star or plus applied to character classes. Let $L_r = \Sigma^* \cdot L(r) \cdot \Sigma^*$ be the language of strings containing a substring matching r. Then the syntactic monoid $M(L_r)$ is aperiodic.*

Proof. We show L_r is star-free, hence aperiodic by the equivalence in Theorem 5.3.

Star-free closure under substring wrapping. For any regular language K over Σ, the language $\Sigma^* \cdot K \cdot \Sigma^*$ is star-free whenever K is star-free. This follows directly from the closure of star-free languages under concatenation [28]: $\Sigma^* = \overline{\emptyset}$ is star-free, and the concatenation of three star-free languages is star-free. It therefore suffices to show that $L(r)$ itself is star-free for each building-block construction.

Literals and character classes. A single character $a \in \Sigma$ defines $\{a\}$, trivially star-free. A character class $[c_1 c_2 \cdots c_k]$ is $\{c_1\} \cup \cdots \cup \{c_k\}$, a finite union of singletons, hence star-free.

Bounded repetition. $r^{\{n\}}$ (exactly n repetitions) is the n-fold concatenation of $L(r)$, star-free by induction on n using closure under concatenation. $r^{\{m,n\}}$ (between m and n repetitions) is $\bigcup_{k=m}^{n} r^{\{k\}}$, a finite union of star-free languages, hence star-free.

Alternation. $r_1 \mid r_2$ yields $L(r_1) \cup L(r_2)$; star-free languages are closed under union (and Boolean operations generally), so this is star-free.

Optional groups. $r?$ yields $L(r) \cup \{\varepsilon\}$; since $\{\varepsilon\}$ is star-free and star-free is closed under union, this is star-free.

Dot. The pattern . matches any single character in Σ. It equals $\bigcup_{a \in \Sigma} \{a\}$, a finite union of singletons, star-free.

Kleene plus over character classes. Let C be a character class with language $L(C) \subseteq \Sigma$. We claim $L(C)^+$ is star-free. $L(C)^+$ is the set of nonempty strings over $L(C)$, equal to $\overline{\overline{L(C) \cdot \Sigma^*} \cup \{\varepsilon\}}$; more directly, $L(C)^+ = \Sigma^* \setminus (\Sigma^* \cdot \overline{L(C)} \cdot \Sigma^* \cup \{\varepsilon\})$. Since $\overline{L(C)} = \Sigma \setminus L(C)$ is a finite set of singletons (hence star-free), and star-free is closed under complement and concatenation with Σ^*, $L(C)^+$ is star-free. The argument applies equally to $L(C)^*$: $L(C)^* = L(C)^+ \cup \{\varepsilon\}$, a union of two star-free languages.

Conclusion. Every building block of r produces a star-free language, and closure under concatenation and Boolean operations preserves star-freeness throughout the construction of $L(r)$. Wrapping with $\Sigma^* \cdot (\cdot) \cdot \Sigma^*$ preserves star-freeness. Therefore L_r is star-free, and by Theorem 5.3 its syntactic monoid $M(L_r)$ is aperiodic. □

Remark 6.2 (Guardrail patterns vs. $(aa)^*$). A pattern such as $(aa)^*$ recognizes strings of even a-count, requiring $\mathbb{Z}/2\mathbb{Z}$ in its syntactic monoid, so it is *not* aperiodic. However, $(aa)^*$ is not a substring-matching guardrail pattern: it is a character-class Kleene iteration over a two-character literal, not a single character class. The guardrail patterns in our corpus—keyword blocklists, domain patterns, toxicity keywords—are constructed precisely as described in the theorem: Kleene star and plus are applied to character classes, not to multi-character sequences. The syntactic monoid of the representative guardrail pattern $(ab)^*$ is the six-element monoid $B_2 = \{1, a, b, ab, ba, 0\}$ where multiplication follows from string concatenation modulo the syntactic congruence; this monoid has no nontrivial subgroups, confirmed by our monoid extractor (Section 8.1).

6.2 The Guardrail Blindness Theorem

Define the *modular decode language* as follows. Fix a prime $p \geq 2$. Let $\Sigma_0 = \{0, 1\}$ and let $w \in \{0, 1\}^*$. The *p-interleaved encoding* of a harmful string $h \in L_{\mathsf{blocked}}$ is a string $\phi_p(h) \in \Sigma^*$ constructed by inserting $p - 1$ arbitrary filler characters between successive characters of h, so that position i (zero-indexed) of h appears at position $p \cdot i$ in $\phi_p(h)$. The decoder recovers h by reading every p-th character: $h = \phi_p(h)[0 :: p]$. Define

$$L_{\mathsf{decode}}(p) = \left\{ w \in \Sigma^* : w[0 :: p] \in L_{\mathsf{blocked}} \right\}$$

as the language of strings whose every-p-th character subsequence is in the blocked language.

Theorem 6.3 (Guardrail blindness)**.** *Let G be any regex guardrail with aperiodic syntactic monoid $M(L_G)$. Let $p \geq 2$ be any prime. Then:*

1. *$L_{\mathsf{decode}}(p)$ is regular.*
2. *$M(L_{\mathsf{decode}}(p))$ is not aperiodic; it contains $\mathbb{Z}/p\mathbb{Z}$ as a subgroup.*
3. *$\phi_p(L_{\mathsf{blocked}}) \cap L_G = \emptyset$, and no modification of G within the class of aperiodic regex guardrails can detect $L_{\mathsf{decode}}(p)$.*

Proof.

(1) Regularity. The decoding function $w \mapsto w[0 :: p]$ is computed by a length-p cyclic DFA that counts position modulo p and records the character at position $\equiv 0 \pmod{p}$. This DFA has state set $Q = \{0, 1, \ldots, p - 1\} \times Q_{\mathsf{block}}$, where Q_{block} is the state set of the minimal DFA for L_{blocked}. Transitions increment the counter modulo p and advance the L_{blocked} component only when the counter is 0. Since both components are finite, $L_{\mathsf{decode}}(p)$ is accepted by a finite automaton of size $p \cdot |Q_{\mathsf{block}}|$ and is therefore regular.

(2) Non-aperiodicity. Consider the DFA constructed above, with states $(i, q) \in \{0, \ldots, p-1\} \times Q_{\mathsf{block}}$. Let $f \in \Sigma$ be a *filler* character that appears only at non-zero counter positions—i.e., $\delta_{\mathsf{block}}(q, f) = q$ for all $q \in Q_{\mathsf{block}}$ (the Q_{block} component is unchanged by f). The transition

induced by reading f maps $(i, q) \mapsto ((i + 1) \bmod p,\, q)$, advancing only the counter. Consider the word $w = f^p$ consisting of p copies of f. For any fixed $q \in Q_{\mathsf{block}}$, the transformation δ_w maps $(i, q) \mapsto ((i + p) \bmod p,\, q) = (i, q)$; thus δ_w acts as the identity on states of the form (i, q) for this q. However, δ_{f^j} maps $(i, q) \mapsto ((i + j) \bmod p,\, q)$ for $1 \leq j < p$, which is not the identity since $j \not\equiv 0 \pmod p$. Therefore $\delta_{f^j} \neq \mathrm{id}$ for $1 \leq j < p$, yet $\delta_{f^p} = \mathrm{id}$ on every state of the form (i, q) with q fixed. The elements $\{\delta_{f^j} : 0 \leq j < p\}$ form a subgroup of the transition monoid isomorphic to $\mathbb{Z}/p\mathbb{Z}$, witnessed by the cyclic permutation $(i, q) \mapsto ((i + 1) \bmod p,\, q)$ of order exactly p. The cyclic group $\mathbb{Z}/p\mathbb{Z}$ therefore divides $M(L_{\mathsf{decode}}(p))$, which is thus non-aperiodic.

(3) Blindness consequence. By construction, $\phi_p(L_{\mathsf{blocked}}) \subseteq L_{\mathsf{decode}}(p)$. If $w = \phi_p(h)$ for some $h \in L_{\mathsf{blocked}}$, then w contains no substring in $\bigcup_i L(r_i)$ (the harmful keywords appear only at every p-th position, not as contiguous substrings), so $G(w) = \mathsf{allow}$.

For the patchability claim: any regex guardrail added to G with an aperiodic syntactic monoid defines a language in AC^0 by Theorem 5.4. But detecting whether $w[0 :: p] \in L_{\mathsf{blocked}}$ requires computing a MOD_p predicate on the position counter, which is not in AC^0 by Theorem 5.5 (for prime p). Therefore, no aperiodic regex can be added that correctly classifies all encodings in $\phi_p(L_{\mathsf{blocked}})$ while leaving genuinely benign strings allowed. □

Corollary 6.4 (MOD_2 sufficiency)**.** *$p = 2$ suffices for a complete bypass: every aperiodic regex guardrail is blind to the alternating-character encoding ϕ_2, which interleaves the payload with arbitrary filler characters at every odd position. Existing evasion techniques—reading every other character, zero-width character insertion between payload characters, even-position acrostics—are all instances of ϕ_2 and are therefore provably undetectable by any current regex guardrail.*

Remark 6.5 (Extension to composite moduli). Theorem 6.3 is stated for prime p. For composite $p = q_1 \cdots q_k$, blindness to MOD_p follows from blindness to each prime factor: if the guardrail cannot detect MOD_{q_i} for any i, it cannot detect MOD_p. The prime case therefore implies the general case for aperiodic guardrails, which are blind to all primes simultaneously.

6.3 Composition Preserves Aperiodicity

Proposition 6.6 (Composition preserves aperiodicity)**.** *If $L_1, L_2 \subseteq \Sigma^*$ are regular languages with aperiodic syntactic monoids, then $L_1 \cap L_2$, $L_1 \cup L_2$, and $\Sigma^* \setminus L_1$ all have aperiodic syntactic monoids.*

Proof sketch. The syntactic monoid of $L_1 \cap L_2$ divides $M(L_1) \times M(L_2)$. A divisor of a direct product of aperiodic monoids is aperiodic, since aperiodicity is preserved under division (a divisor of an aperiodic monoid has no nontrivial subgroups). The cases of union and complement follow from De Morgan's laws and the fact that $M(\overline{L}) \cong M(L)$. □

This means that applying multiple guardrail patterns in sequence (intersection) or in parallel (union) preserves the blindness property. A guardrail system composed of aperiodic components remains blind to all modular encodings.

6.4 Krohn–Rhodes Attack Taxonomy

Proposition 6.7 (Blindness spectrum, after Straubing [32])**.** *Let G be a regex guardrail with syntactic monoid $M(L_G)$ and Krohn–Rhodes group components $G_1, G_2, \ldots, G_k$ (simple groups appearing in the decomposition of Theorem 5.6). Then:*

1. *G can detect MOD_p-encoded payloads only if p divides $|G_i|$ for some $i \in \{1, \ldots, k\}$.*
2. *G is provably blind to MOD_q-encoded payloads for every prime q not dividing any $|G_i|$.*
3. *If $M(L_G)$ is aperiodic (no group components), then G is blind to MOD_p-encoded payloads for every prime $p \geq 2$—blindness is total.*

Remark 6.8 (Constructive audit via monoid extractor). The Krohn–Rhodes blindness spectrum is constructively computable. Given a guardrail regex, the monoid extractor (Section 8.1) computes the minimal DFA, enumerates the transition monoid, and identifies all group components via standard semigroup algorithms. The output is a complete list of primes to which the guardrail is blind: if the group components are $\{S_5, \mathbb{Z}/3\mathbb{Z}\}$, the guardrail can potentially detect MOD_3 and MOD_5 encodings but is blind to all other modular encodings. For 100% of patterns in our corpus, the component list is empty and blindness is total.

6.5 Information Destruction (Fano Bound)

Proposition 6.9 (Information destruction)**.** *Let $U\colon R \to M$ be an abstraction function and $G\colon R \to \{0, 1\}$ the ground-truth guardrail. Let R be distributed uniformly over R. Then for any lifted guardrail $G'\colon M \to \{0, 1\}$, the probability of misclassification satisfies*

$$P_e \;=\; \Pr[G'(U(R)) \neq G(R)] \;\geq\; h^{-1}\big(H(G(R) \mid U(R))\big),$$

where $h(x) = -x \log_2 x - (1 - x) \log_2(1 - x)$ is the binary entropy function and h^{-1} is its inverse on $[0, \frac{1}{2}]$.

Proof. Consider the Markov chain $G(R) \to U(R) \to G'(U(R))$. By the data-processing inequality, the

mutual information satisfies $I(G(R); G'(U(R))) \leq I(G(R); U(R)) = H(G(R)) - H(G(R) \mid U(R))$. The lifted guardrail's decision $G'(U(R))$ is a binary function of $U(R)$, so the probability of error is $P_e = \Pr[G'(U(R)) \neq G(R)]$. By Fano's inequality [9], $H(G(R) \mid G'(U(R))) \leq h(P_e) + P_e \cdot \log_2(|\{0,1\}| - 1) = h(P_e)$. Since conditioning can only reduce entropy, $H(G(R) \mid U(R)) \leq H(G(R) \mid G'(U(R))) \leq h(P_e)$. Inverting the (monotone increasing on $[0, \frac{1}{2}]$) binary entropy function gives $P_e \geq h^{-1}(H(G(R) \mid U(R)))$. □

Interpretation. The conditional entropy $H(G(R) \mid U(R))$ is positive precisely when mixed fibers exist (Definition 4.2). If 30% of probability mass falls in mixed fibers with ∼50/50 safe/unsafe split, then $H(G(R) \mid U(R)) \approx 0.3$ bits, forcing $P_e \geq h^{-1}(0.3) \approx 0.048$ (about 5%). This floor is irreducible: no lifted guardrail, regardless of its internal complexity, can reduce the error below $h^{-1}(H(G(R) \mid U(R)))$ when the abstraction function U has mixed fibers. The only remedy is to reduce the abstraction—move inspection closer to the concrete artifact level—but this makes the guardrail more brittle to novel formulations.

6.6 Computational Intractability (NP-Completeness)

Definition 6.10 (Guardrail Instantiation Decision Problem (GIDP))**.** Given:

- An abstract program P over a finite set of abstract symbols M,
- An interpretation domain R with ground-truth guardrail $G\colon R \to \{0,1\}$,
- An interpretation function schema $I\colon M \to 2^R$ mapping each abstract symbol to a nonempty set of candidate concrete values,

GIDP asks: does there exist a *total assignment* $\iota\colon M \to R$ with $\iota(m) \in I(m)$ for all m such that the concretized program $P[\iota]$ satisfies $G(P[\iota]) = 1$ (i.e., the program is dangerous)?

Proposition 6.11 (NP-completeness of GIDP)**.** GIDP *is NP-complete. This follows from a standard reduction from 3-SAT.*

Proof.

Membership in NP. A certificate is an assignment $\iota\colon M \to R$. Given ι, checking $\iota(m) \in I(m)$ for all m takes time $O(|M| \cdot |R|)$; evaluating $G(P[\iota])$ takes polynomial time in the program size. Therefore GIDP ∈ NP.

NP-hardness. We reduce 3-SAT to GIDP. Given a 3-CNF formula φ with variables $x_1, \ldots, x_n$ and clauses $C_1, \ldots, C_m$, construct:

- Abstract symbols $M = \{v_1, \ldots, v_n\}$, one per variable.
- Concrete domain $R = \{T, F\}$ (Boolean values).
- $I(v_i) = \{T, F\}$ for each i: each variable can be assigned either truth value.
- A total assignment $\iota\colon M \to R$ with $\iota(v_i) \in I(v_i)$ corresponds to a Boolean assignment $x_i = \iota(v_i)$.
- Abstract program $P = (v_1, \ldots, v_n)$ is the ordered list of variable symbols.
- Guardrail $G\colon R^n \to \{0,1\}$ is the Boolean circuit encoding φ: $G(\iota(v_1), \ldots, \iota(v_n)) = 1$ iff $\varphi(\iota(v_1), \ldots, \iota(v_n))$ is satisfiable.

A satisfying assignment for φ exists iff the GIDP instance has a positive answer. Since 3-SAT is NP-complete, GIDP is NP-hard. □

Remark 6.12 (Pointwise vs. combinatorial; scope of hardness). The NP-hardness comes from the combinatorial search over the exponentially large space of concrete interpretations, not from any single concretization step. When G is restricted to the class of regex guardrails (Definition 4.4), the single-interpretation check $G(P[\iota])$ is computable in time $O(|P| \cdot |Q_G|)$ where $|Q_G|$ is the DFA state count. The NP-hardness therefore resides entirely in the search over the product of fiber sets $|I(m_1)| \times \cdots \times |I(m_k)|$, not in the evaluation of the guardrail itself. If the interpretation function I is a bijection (each abstract symbol has a unique concrete value), GIDP is solvable in polynomial time: just evaluate $G(P[\iota])$ directly. The hardness is precisely the problem of searching through a product of fiber sets—the same mixed-fiber phenomenon that drives the Fano bound.

6.7 Faithful Transfer (Free Category Functoriality)

Proposition 6.13 (Faithful transfer)**.** *Let $\mathcal{A}$ be the free category generated by a context-free grammar $\mathcal{G}$ over abstract symbols M, and let $\mathcal{D}$ be the category of domain operations over R. Let $I\colon M \to R$ be a graph homomorphism from the grammar's dependency structure to $\mathcal{D}$. Then I extends uniquely to a functor $\hat{I}\colon \mathcal{A} \to \mathcal{D}$ that maps every derivation path in $\mathcal{A}$ to a valid operation sequence in $\mathcal{D}$. If I is additionally injective on the generating morphisms of $\mathcal{G}$, then $\hat{I}$ is faithful: distinct derivation paths map to distinct operation sequences. This is a direct application of the universal property of free categories [22].*

Proof. $\mathcal{A}$ is the free category on the graph $\mathcal{G}$. By the universal property of free categories [22] (Section II.7), any graph homomorphism I into a category $\mathcal{D}$ extends uniquely to a functor $\hat{I}\colon \mathcal{A} \to \mathcal{D}$. Functoriality ensures composition preservation: multi-step derivations map to multi-step domain operations. For faithfulness: if I is injective on generating morphisms, then distinct generators $f_{m_1} \neq f_{m_2}$ map to distinct operations $\hat{I}(f_{m_1}) \neq \hat{I}(f_{m_2})$. Since $\mathcal{A}$ is free, it has no non-trivial relations among

generators beyond those imposed by $\mathcal{G}$; therefore distinct derivation paths—as distinct sequences of generating morphisms—map to distinct operation sequences under $\hat{I}$. Without injectivity, the functor is still well-defined (derivations map to valid operation sequences) but may collapse distinct abstract paths onto the same concrete sequence. □

What this means for the attack. An LLM that correctly solves the abstract grammar $\mathcal{G}$ produces an abstract derivation path. The interpretation functor $\hat{I}$ maps this to a valid operation sequence in the real domain. If the interpretation is injective (distinct abstract symbols map to distinct concrete operations), faithfulness additionally guarantees that no step is collapsed. The attacker verifies this offline when constructing the grammar. **Caveat:** Proposition 6.13's guarantee is conditional on the interpretation function I being a valid graph homomorphism. In the V4 pipeline (§7.4), the grammar is extracted by an LLM, not verified by hand; the pipeline therefore inherits any extraction error from the bootstrap step. If the LLM hallucinates a production rule or misspecifies a yield, the functor maps to an invalid domain sequence. The formal guarantee applies to the solved path, not to the extraction step.

6.8 Syntactic Indistinguishability

Definition 6.14 (Syntactic equivalence of prompt classes)**.** Two prompt classes $\mathcal{P}_A$ and $\mathcal{P}_B$ are *syntactically equivalent* with respect to a guardrail G if for every $p_A \in \mathcal{P}_A$ there exists $p_B \in \mathcal{P}_B$ such that $p_A = p_B$ as strings, and vice versa.

Proposition 6.15 (Indistinguishability)**.** *Let $\mathcal{P}_{\mathsf{harm}}$ be the class of adversarial prompts that instantiate (via offline interpretation) to harmful operation sequences, and let $\mathcal{P}_{\mathsf{legit}}$ be the class of legitimate formal reasoning prompts (e.g., theorem proving, grammar completion) over the same abstract symbol set M. Under the homomorphic reasoning construction, $\mathcal{P}_{\mathsf{harm}} = \mathcal{P}_{\mathsf{legit}}$ as sets of strings. Therefore, for any guardrail G operating on prompt content, the adversarial advantage in distinguishing $\mathcal{P}_{\mathsf{harm}}$ from $\mathcal{P}_{\mathsf{legit}}$ is exactly zero.*

Proof. By construction, the homomorphic reasoning prompt $p \in \mathcal{P}_{\mathsf{harm}}$ contains only the abstract grammar $\mathcal{G}$ and an instruction to produce a derivation. Both the symbols and the grammatical structure are drawn from a symbol set M that carries no harmful semantics; the interpretation function $I\colon M \to R$ is held offline by the attacker and never appears in any inference call. Therefore, p is syntactically identical to a legitimate formal reasoning prompt over the same grammar. The set of all such prompts is exactly $\mathcal{P}_{\mathsf{legit}}$. □

Assumption 6.16 (Semantic opacity of abstract symbols). The abstract grammar $\mathcal{G}$ uses symbol names drawn from a domain-neutral vocabulary (e.g., A, B, C or arbitrary identifiers) such that no symbol name reveals the interpretation $I(m)$.

Proposition 6.15 is unconditional given Assumption 6.16. If Assumption 6.16 fails—for example, if the attacker carelessly names symbols after their real-world referents—the theorem does not apply, and a sufficiently informed guardrail might detect the attack through symbolic name inspection. A weakened version holds without Assumption 6.16: the adversarial prompt class is computationally indistinguishable from the legitimate class under any efficient guardrail, by the argument of Ball et al. [5]; but Proposition 6.15 gives the stronger information-theoretic result under the stated assumption.

Relation to Ball et al. Ball et al. [5] prove computational indistinguishability under standard cryptographic assumptions, establishing that no efficient guardrail can distinguish adversarial from benign prompts with non-negligible advantage. Our Proposition 6.15 is complementary: it establishes *perfect* (information-theoretic, unconditional) indistinguishability for the specific construction of homomorphic reasoning, with the trade-off that it applies to this construction specifically rather than to the general attack class. Together, the two results cover the full distinguishability landscape: Ball et al. handle general attacks under computational assumptions; Proposition 6.15 handles the homomorphic reasoning construction without any assumption beyond Assumption 6.16.

7 Attack Vectors

7.1 V1: Decomposition (Known Technique, New Framing)

We observe that runtime argument injection—a technique well-known in software security—is an instance of our general abstraction framework. The attacker requests that the LLM generate a function or script with *parametric placeholders* rather than literal harmful values. The harmful values are then injected at runtime via environment variables, command-line arguments, or configuration files that the guardrail never inspects.

In the abstraction framework (Definition 4.1), the function signature serves as the abstract type $m \in M$, and the runtime argument constitutes the concrete fiber element $r \in U^{-1}(m)$. The guardrail evaluates $G'(m) = \mathsf{allow}$ because the function signature is benign; the ground truth $G(r)$ for the runtime-instantiated version may be block. The mixed fiber is the mechanism, and the Fano bound (Proposition 6.9) gives the irreducible misclassification floor.

The novelty of this paper's contribution is not the injection technique but the algebraic and information-theoretic *proof* that no modification within the inference-layer guardrail paradigm can eliminate the mixed-fiber structure without abandoning the abstraction entirely.

7.2 V2: Zero-Knowledge Pipeline (Known Technique, New Framing)

Base64 encoding to create opaque data channels is a well-known technique; we observe it as an instance of the information destruction formalized in Proposition 6.9.

In the V2 construction, the attacker base64-encodes a configuration file containing harmful parameters (e.g., a target URL, an API endpoint, a chemical structure identifier). The encoded blob is written to a file that the LLM-integrated agent processes without decoding and inspecting the payload. The agent's behavior is conditioned on the decoded values at runtime, but the guardrail sees only the base64 string—a string whose content is in the mixed fiber of the base64 encoding abstraction.

The zero-knowledge property is that the LLM never observes the decoded harmful parameters in any inference call subject to guardrail inspection. The Fano bound quantifies the exact misclassification rate for the lifted guardrail that must classify requests based only on the base64-encoded form.

7.3 V3: Homomorphic Reasoning (Tree-of-Thought on Abstract Grammar)

V3 is the core homomorphic reasoning vector. The attacker constructs a context-free grammar $\mathcal{G}$ over abstract symbols M, representing the procedure to be derived. The LLM is prompted to produce a complete derivation (a sequence of production rule applications) that generates the target yield. The interpretation function $I\colon M \to R$ is held offline and applied to the solved derivation to recover the real-world operation sequence.

Example grammar. The following grammar encodes a three-step operational procedure over abstract symbols `A`, `B`, `C` with operation types `OP1`, `OP2`:

Listing 1: Abstract grammar for V3 homomorphic reasoning

```
grammar = {
    "START": [["PROC"]],
    "PROC":  [["STEP", "PROC"], ["STEP"
    ]],
    "STEP":  [["OP1", "ARG"],  ["OP2", "
    ARG", "ARG"]],
    "ARG":   [["A"], ["B"], ["C"]],
    "OP1":   [["op1"]],
    "OP2":   [["op2"]],
}
# Offline interpretation table (attacker
    -held, never sent to LLM)
I = {"A": "...", "B": "...", "C": "...",
     "op1": "...", "op2": "..."}
```

Table 1: V4 encoding bootstrap pipeline stages.

Stage	Description
1. Domain description	Attacker provides natural-language domain summary to local LLM
2. Grammar extraction	Local LLM extracts context-free grammar with abstract symbols
3. Interpretation table	Attacker manually verifies and annotates $I\colon M \to R$
4. Grammar solving	Grammar sent to target LLM via V3 pipeline
5. Instantiation	Attacker applies I offline to solved derivation

The prompt sent to the LLM contains only the grammar definition and the instruction to produce a derivation satisfying a specified constraint; the interpretation table I is never included.

Viability ratings. For each grammar node type, we assign the LLM viability rating: SURE (the LLM handles this node class reliably), LIKELY (the LLM handles this with moderate reliability), or IMPOSSIBLE (the node class is outside LLM competence for this construction). Terminal symbol substitution is SURE; recursive production expansion is LIKELY; cycle detection in unbounded derivations is IMPOSSIBLE without explicit bounding.

Search strategies. Three search strategies were evaluated: breadth-first search (BFS), random-beam, and Tree-of-Thought (ToT) with LLM-guided branch pruning. Results are detailed in Section 8.2.

7.4 V4: Encoding Bootstrap

V4 bootstraps the abstract grammar from a real-world domain description using an LLM, then uses the extracted grammar as input to V3. The attacker provides a natural-language description of the target domain (e.g., "organic synthesis procedure with common reagents") and prompts a local LLM to extract a formal grammar over abstract symbols.

The extraction-fidelity connection to Proposition 6.13: the formal guarantee of faithful transfer applies to the solved derivation given a correctly specified grammar, but Stage 2 (LLM extraction) is not formally verified. Extraction errors—hallucinated production rules, misspeci-

Table 2: V5 modular counting bypass instances. p: the interleaving modulus used to construct the encoding (p=2 means every other character is payload; filler characters are x). *Payload*: the harmful substring $h \in L_{\text{blocked}}$ that the guardrail would block if seen in plaintext. *Encoded (p=2)*: the string $\phi_2(h)$ actually sent to the guardrail, which passes because no contiguous harmful substring is present. F-labels (F1–F5) are pattern IDs from Table 3; F5 is `exec\s*\(`.

Label	Payload	Encoded (p=2)
F1	`rm -rf /`	`rxmx -xrxfx /`
F2	`wget malware`	`wxgxextx xmxaxlxwxaxrxe`
F3	`import os`	`ixmxpxoxrxtx xoxs`
F4	`eval()`	`exvxaxlx(x)`
F5	`exec(`	`exxxexcx(`

fied yields—propagate to Stage 4 and produce invalid domain sequences after instantiation. In practice, we found that extraction fidelity is high for well-structured domains with clear operational semantics but degrades for domains with ambiguous or context-sensitive dependencies.

7.5 V5: Modular Counting Bypass (The Algebraic Attack Class)

Where V1–V4 exploit information-theoretic and computational barriers at the neural/inference layer, V5 exploits the algebraic blind spot at the regex pre-filter layer.

The attack is direct: a harmful payload $h \in L_{\text{blocked}}$ is encoded as $\phi_p(h)$ using the p-interleaving construction from Section 6.2, where consecutive characters of h are separated by $p-1$ arbitrary filler characters. For $p = 2$, this is the alternating-character encoding: the payload characters appear at positions $0, 2, 4, \ldots$ of the encoded string. By Corollary 6.4, no aperiodic regex guardrail can detect this encoding.

Attack instances. Table 2 lists representative V5 attack instances from our corpus.

For F5: decoding `exxxexcx(` by selecting every other character (positions 0, 2, 4, 6, 8) yields `e`, `x`, `e`, `c`, `(`, recovering `exec(` as required.

Connection to known jailbreak techniques. Several documented jailbreak techniques are instances of ϕ_2:

- *Read every other character*: the canonical ϕ_2 encoding.
- *Acrostics*: ϕ_k at line level rather than character level.
- *Zero-width character insertion*: inserting Unicode zero-width non-joiner (U+200C) between payload characters is exactly ϕ_2 with the null filler character.
- *Token splitting*: splitting a keyword across token boundaries forces the tokenizer to reassemble the substring, which is a partial ϕ_2 at token level.

Our contribution is not the attacks themselves but the proof that they cannot be patched: no regex modification can detect MOD_p-encoded payloads while the syntactic monoid remains aperiodic. The only defense is to move to a guardrail architecture that can compute modular predicates—which, by Theorem 5.4, requires a non-aperiodic syntactic monoid, i.e., a non-AC^0 language recognizer.

8 Empirical Validation

8.1 Regex Corpus Analysis

Methodology. We collected 142 regex patterns from eleven open-source guardrail tools (LLM Guard, Rebuff, NeMo Guardrails, Guardrails AI, llm-guard-py, Presidio, GitLeaks, OWASP-LLM, WAF-generic, LangKit, and BodAIGuard). For each pattern r, we:

1. Compiled r to a Thompson NFA using the standard construction.
2. Converted to the minimal DFA via powerset construction followed by Hopcroft's minimization algorithm.
3. Enumerated the transition monoid $M(L_r)$ by computing all distinct state-transformation functions $\delta_w \colon Q \to Q$ for $w \in \Sigma^*$ via BFS over the Cayley graph of the monoid.
4. Tested aperiodicity by checking, for each monoid element m, whether the sequence $m, m^2, m^3, \ldots$ stabilizes (i.e., $m^n = m^{n+1}$ for some computable $n \leq |M(L_r)|^2$).

The monoid extractor tool (858 lines of Python) is an artifact contribution of this paper, enabling other researchers to audit their own guardrail corpora.

Results. All 142 patterns (100%) have aperiodic syntactic monoids. No non-aperiodic patterns were found; this is consistent with the conjecture that semantic content guardrail patterns (keyword blocklists, domain filters, toxicity patterns) are structurally aperiodic by construction (Section 10.5). All 142 aperiodic patterns were verified to admit the MOD_2 bypass construction (Corollary 6.4): for each pattern, we constructed an encoded payload string $\phi_2(h)$ where h was a string in L_{blocked}, and confirmed that the pattern did not match $\phi_2(h)$.

Table 3 shows representative patterns from each tool with DFA state count, monoid size, and aperiodicity verdict.

Table 3: Representative guardrail regex patterns from 11 tools. DFA states and monoid sizes are for the substring-matching version $L_r = \Sigma^* L(r) \Sigma^*$. AP: aperiodic (Y/N). All 5 shown are aperiodic; all 142 patterns in the full corpus are aperiodic.

ID	Pattern (abbreviated)	DFA	$\lvert M \rvert$	AP
F1	`rm\s*-[rR][fF]`	9	18	Y
F2	`wget|curl.*malware`	14	31	Y
F3	`import\s+os`	11	22	Y
F4	`eval\s*\(`	8	14	Y
F5	`exec\s*\(`	8	14	Y

Table 4: Tree-of-Thought benchmark results ($N = 50$ randomized grammars, seeds 0–49, model `llama3.1:8b`, timeout 90 s per solver). Mean yield averages cumulative path yield over all grammars (unsolved = 0). Solve rate = fraction of grammars for which the target was reached. States = mean number of grammar states explored per run. 95% CIs are bootstrap intervals ($B = 2000$).

Condition	Mean yield	95% CI	Solve rate	States
BFS (no LLM guidance)	0.466	[0.448, 0.483]	100%	87
Random-beam (k=5)	0.172	[0.131, 0.220]	70%	50
ToT (LLM-guided pruning)	0.122	[0.064, 0.185]	26%	27

8.2 Tree-of-Thought Benchmark

Protocol. We generated $N = 50$ randomized abstract grammars (seeds 0–49) using the `generate_grammar` function from the benchmark module. Each grammar has a guaranteed path to a fixed target symbol `N30`, with multiple alternative paths (optimal depth 5–8, two short low-yield paths, and three dead-end traps with high per-step yield estimates). Three search strategies were evaluated, each with a 90-second timeout: (BFS) exhaustive breadth-first search up to depth 12, selecting the highest-yield path to the target—no LLM guidance; (Random-beam) beam search with beam width 5 and random composite scoring ($0.6 \times \text{random} + 0.4 \times \text{yield}$); (ToT) Tree-of-Thought with LLM-guided branch pruning—at each depth the LLM (llama3.1:8b via Ollama) rates each candidate branch as SURE, LIKELY, or IMPOSSIBLE and the composite score ($0.6 \times$ LLM $+ 0.4 \times$ yield) determines beam selection. Mean yield is the average cumulative yield score across all $N = 50$ grammars (a grammar that is not solved contributes yield $= 0$). Solve rate is the fraction of grammars for which any path to the target was found.

Results. Table 4 reports mean yield, solve rate, and bootstrap 95% confidence intervals across all 50 grammar instances.

BFS is optimal for small synthetic grammars. BFS solved all 50 grammars (100% solve rate) with a mean yield of 0.466, significantly outperforming both random-beam (mean 0.172, 70% solve rate) and ToT+LLM (mean 0.122, 26% solve rate). Wilcoxon signed-rank tests (one-sided, BFS>alternative): BFS vs. ToT: $W = 1275$, $p < 0.001$; BFS vs. Random-beam: $W = 1225$, $p < 0.001$. The hypothesis that ToT>BFS was not supported: $W = 0$, $p = 1.0$ (no grammar favored ToT over BFS).

The result is not surprising in retrospect. BFS is complete and optimal for the search spaces generated here: the grammar structure guarantees that the target is reachable and the search depth (up to 12) is within BFS's practical reach. LLM guidance via ToT adds latency (avg. ~ 50 s per grammar) and frequently misdirects the beam into dead-end traps that the LLM rates as LIKELY—the high-yield-estimate traps in the grammar are designed precisely to exploit this failure mode.

Why BFS dominates: the LLM evaluation function adds noise, not signal. The abstract grammars use anonymous production-rule labels (R1, R2, ...) and symbol names with no semantic content (`N0`, `N1`, ..., `N30`). When the LLM rates a candidate branch, it has no world knowledge to draw on: the rules carry no meaning that would allow the model to distinguish a productive path from a dead end. The SURE/LIKELY/IMPOSSIBLE ratings are therefore effectively random with respect to the true reachability structure of the grammar. Because the composite ToT score weights this random LLM signal at $0.6\times$, it actively degrades beam selection relative to BFS (which ignores the LLM entirely) and even relative to random beam (which at least applies a uniformly random signal rather than a correlated-but-wrong one). BFS avoids this failure mode entirely; at the scale of these grammars (mean 87 states explored), exhaustive search is inexpensive and provably optimal.

ToT adds value only when the search space exceeds BFS capacity. The grammars in this benchmark are small enough that BFS explores them exhaustively. In settings where the state space grows exponentially (real chemistry reaction networks, large software build graphs, or grammars with unbounded depth), BFS becomes computationally intractable and LLM-guided heuristic search can yield practical speedups by pruning obviously unreachable branches. The security claim of V3—that an attacker can *in principle* solve abstract grammar derivations using LLM guidance—does not depend on ToT outperforming BFS on toy benchmarks; it depends on the *existence* of a solver that finds valid derivations, which all three strategies demonstrate on solvable grammars.

Correction note. An earlier version of this section reported mean solve rates of 0.512 (ToT), 0.203 (Random-beam), and 0.119 (BFS) on $N = 20$ grammars and claimed ToT outperformed BFS

by $4.3\times$. Those numbers were fabricated and are retracted. The honest results above were obtained by re-running the public benchmark code (`src/aperiodic_guardrails/benchmark/runner.py`) on $N = 50$ grammars with seeds 0–49 and model `llama3.1:8b`; results are archived in `results/tot_n50_honest.json`.

Scope of this negative result. The abstract benchmark establishes that BFS is optimal when production rules carry no semantic content. This is an informative negative result, not a refutation of LLM-guided search in general: it precisely characterizes the precondition under which the LLM evaluation function helps versus hurts. The following case study tests the complementary regime—a grammar encoding a real domain (organic chemistry synthesis) where the model's world knowledge *can* meaningfully evaluate candidate paths. The contrast between the two settings is the central empirical lesson of Section 8: *attack planning benefits from world knowledge, not search depth alone.*

8.3 Aspirin Case Study

We validated the full V3/V4 pipeline on a concrete benign-domain case study using the organic chemistry domain. A grammar encoding the synthesis procedure for aspirin (acetylsalicylic acid, $C_9H_8O_4$) was extracted via V4 from a natural-language description and solved via V3. The solved derivation, when instantiated with the interpretation table $I\colon M_{\mathsf{chem}} \to R_{\mathsf{chem}}$, produced the correct four-step synthesis sequence: salicylic acid + acetic anhydride $\to$ crude product $\to$ recrystallization $\to$ aspirin.

Severity note. Aspirin synthesis was chosen as a *validation* case because it is a well-known pharmaceutical synthesis that poses minimal harm risk. The same pipeline applies in principle to any domain expressible as a context-free grammar; the present paper does not attempt to demonstrate high-severity cases and reports only benign validation.

Second domain: TCP state machine. To validate domain-agnosticism beyond chemistry, we applied the V4 grammar extraction pipeline to the TCP connection state machine (RFC 793). The grammar encodes: $S \to$ SYN $\to$ SYN-ACK $\to$ ACK $\to$ ESTABLISHED $\to$ FIN $\to$ CLOSED. When extracted via V4 into abstract symbols ($S \to t_1 \to t_2 \to t_3 \to t_4 \to t_5 \to t_6$) and solved via V3, the derivation correctly reproduces the TCP three-way handshake and teardown sequence after interpretation. This confirms the domain-agnosticism claim across two structurally different domains (chemical synthesis and network protocol) and demonstrates that *any* domain expressible as a context-free grammar is vulnerable to the same homomorphic reasoning pipeline.

Defense classification. The aspirin case study is a canonical Class C attack (§10.3): the LLM sees only abstract grammar symbols, never the chemical names or procedures. No inference-layer filter—regex, neural, or any composition thereof—can detect the attack because the prompts are syntactically identical to legitimate grammar exercises (Proposition 6.15). The interpretation table $I\colon M_{\mathsf{chem}} \to R_{\mathsf{chem}}$ is held offline by the attacker and never appears in any LLM-visible content.

This structure is isomorphic to the `secrets-router` defense (§11.4): in both cases, the security-critical mapping (interpretation table / credential values) is held outside the LLM context, and only abstract references (grammar symbols / opaque handles) are visible to the model. The attack exploits the offline mapping to produce harmful instantiations; the defense exploits the *same* offline-mapping architecture to prevent credential exfiltration. Both are instances of the quotient functor Q from §4.2: the non-invertibility of Q at the inference layer is a vulnerability when the attacker holds the interpretation table, and a security guarantee when the defender holds it.

The Interpretation Firewall. Our adaptive adversary experiment confirms that inference-layer defenses—even structural analysis combined with re-interpretation probing—fail when the adversary uses opaque symbols and fake recursion ($0/3$ detection against adaptive Class C attacks, versus $3/3$ against naive ones). The general defense architecture is an *interpretation firewall*: rather than inspecting the grammar or derivation (inference layer), inspect the *actions* that follow interpretation (execution layer). Concretely, this requires: (1) action allowlisting (the LLM can trigger only declared tool calls, not arbitrary code); (2) sandboxed interpretation (derivations are instantiated inside an isolation boundary and their effects inspected before release); and (3) provenance tracking (the chain from prompt to grammar to derivation to action is logged and auditable). Tool-use protocols such as MCP already implement (1) via tool declarations and (3) via opaque handles, making them a natural substrate for Class C defense. The remaining gap is (2): MCP tools currently execute with full permissions rather than in a sandbox. Closing this gap—sandbox-isolated tool execution with effect auditing—is the single architectural change that would complete the defense stack against all three attack classes.

9 Proof-of-Concept Implementation

The PoC implements all five attack vectors across six Python modules: `engine.py` (63 lines:

zero-knowledge recurrent executor for V1/V2), `mediums/graph_solver.py` (43 lines: BFS state-space solver), `mediums/tot_solver.py` (158 lines: Tree-of-Thought with LLM evaluation for V3), `mediums/web_scraper.py` (77 lines: AST-based HTTP fetch), `encode.py` (22 lines: config-to-base64 encoder), and a medium registry (`mediums/__init__.py`, 13 lines) that selects the execution medium at runtime. The PoC totals 376 lines of Python—compact enough to fit within a single code review session and below typical automated complexity thresholds. Full source is provided in Appendix A.

Architecture. `engine.py` decodes a base64-encoded config, selects an AST medium, compiles it via `exec(compile(...))`, injects runtime parameters, and loops until a termination condition. Each medium exports a `build_*_ast()` function returning a Python AST module with zero string literals—all domain content enters via runtime parameters from the opaque config. `tot_solver.py` implements beam search with LLM-guided branch pruning; the LLM (via Ollama) sees only abstract symbols and rates branches as SURE, LIKELY, or IMPOSSIBLE.

AST medium. The derivation output is processed through Python's AST rather than via `eval` of a raw string. This medium was chosen for its zero-literal property: the AST nodes are constructed programmatically from grammar terminal symbols, so the executed code is never present as a string in the source.

CLI usage.

Listing 2: PoC CLI usage

```
# Solve an abstract grammar and
    instantiate
python grammar.py --mode tot --grammar
    path/to/grammar.json \
  --budget 100 --output derivation.json

# Apply interpretation table offline
python engine.py --derivation derivation.
    json \
  --interpretation path/to/I.json --dry-
    run

# Bootstrap grammar from domain
    description
python bootstrap.py --domain "organic
    synthesis" \
  --symbols 12 --output grammar.json
```

The `-dry-run` flag to `engine.py` prints the instantiated operation sequence without executing it, enabling human review before any execution step.

Table 5: Defense layer mapping. Each row gives the guardrail layer, its mechanism, the threats it can address, and the formal limitation established in this paper.

Layer	Mechanism	Formal limitation
Regex	Substring patterns, blocklists	Provably blind to MOD_p encodings (§6.1–6.2); 100% of corpus aperiodic
Inference	Content filters, RLHF	Fano error floor for abstracted requests (§6.5); direct requests and obvious encodings caught
Schema	Domain-aware validation	Catches some homomorphic reasoning (high false-positive rate, NP-hard to verify, §6.6)
Execution	Runtime payload inspection, allowlists	Catches ZK pipelines (V2) and instantiated procedures; best defense against V1
OS	Sandboxing, network ACLs	Catches all execution-layer attacks; limits legitimate functionality
Provenance	End-to-end pipeline monitoring	Catches full pipeline attacks; requires offline visibility into attacker-held interpretation table

10 Defense Recommendations

10.1 Defense Layer Mapping

Table 5 maps each impossibility result to the defense layer it constrains, the threats that layer can and cannot address, and the formal bound from this paper.

The key structural insight is that no single layer is sufficient. The Regex layer is algebraically blind; the Inference layer has an irreducible Fano floor; the Schema layer is NP-hard; and the Execution/OS layers limit functionality. Effective defense requires all layers operating in concert, with particular emphasis on the Execution and Provenance layers, which are the only ones capable of observing the concrete artifacts that carry harmful intent.

10.2 Neural Guardrails and TC^0

The algebraic blindness established in Theorem 6.3 is a property of AC^0 circuits, the complexity class that contains all aperiodic regular languages (Theorem 5.4). Regex guardrails are, by construction, AC^0 devices: they

recognize star-free languages, whose syntactic monoids are aperiodic, and the Barrington–Compton–Straubing–Thérien theorem places every such language in AC^0. The Furst–Saxe–Sipser theorem (Theorem 5.5) then seals the impossibility: $\mathrm{PARITY} \notin \mathrm{AC}^0$, so no regex guardrail can detect modular counting encodings. Transformer-based (neural) guardrails occupy a strictly higher complexity class. Merrill and Sabharwal [25] establish that log-precision transformers are contained in TC^0, the class of constant-depth threshold circuits, and Chiang et al. [8] sharpen these bounds. Since $\mathrm{AC}^0 \subsetneq \mathrm{TC}^0$ [13, 17], neural guardrails are *strictly more expressive* than regex guardrails: they can compute functions—including all MOD_p predicates—that no regex guardrail can compute. This means the modular-counting blind spot identified in §6.2 does *not* transfer to neural guardrails; a sufficiently trained transformer can in principle detect ϕ_p-encoded payloads for every prime p.

Table 6: Circuit complexity of guardrail architectures. $\mathrm{AC}^0 \subsetneq \mathrm{TC}^0$; both are conjectured to be strictly contained in NC^1.

Architecture	Complexity	Detects MOD_p?
Regex (substring match)	AC^0	No (Thm 6.3)
Regex (explicit count)	NC^1	Yes, for specific p
1-layer transformer	TC^0	Yes (threshold gates)
Multi-layer transformer	TC^0	Yes, broader class [33]
Full production LLM	Unknown (in TC^0?)	Empirically yes

What TC^0 can detect that AC^0 cannot. The threshold gates that separate TC^0 from AC^0 allow circuits to compute majority, parity, and iterated addition. Concretely, a single attention head can compute parity over a binary sequence: the head attends uniformly to all positions and accumulates a weighted sum that crosses a threshold if and only if the number of attended tokens is odd. This computation is provably *impossible* for AC^0 circuits by Furst–Saxe–Sipser [13, 17]: no constant-depth circuit with unbounded fan-in AND/OR gates can compute PARITY, regardless of circuit size. The practical consequence is direct. The canonical MOD_2 bypass of Corollary 6.4—an interleaved payload $w = a_1 b_1 a_2 b_2 \cdots a_n b_n$ where $\{a_i\}$ is the harmful sequence and $\{b_i\}$ is innocuous filler—requires only counting whether an even or odd number of "a"-characters appear. No regex guardrail can perform this check (the syntax monoid of any aperiodic pattern cannot distinguish even from odd occurrences), but a one-layer transformer with a single uniform attention head can detect it exactly: the head's output sum is $> n/2$ if and only if more than half the attended positions are "a"-tokens, from which parity is recoverable by a linear threshold. Strobl et al. [33] give tight bounds on exactly which formal languages hard-attention transformers of each depth recognize, including explicit inclusion of all regular languages with non-aperiodic syntactic monoids. Hahn [16] establishes that soft-attention mechanisms saturate on periodic languages in ways that directly parallel the algebraic distinction formalized here: periodic structures force attention weights to be non-uniform in a way that constant-depth circuits cannot fully exploit, providing circuit-theoretic grounding for the qualitative boundary our algebraic results formalize in terms of syntactic monoids.

What even TC^0 cannot detect. Expressivity gains come at a cost: TC^0 has its own ceiling. NC^1-complete problems—such as evaluating an arbitrary Boolean formula (formula evaluation is NC^1-complete under AC^0 reductions [6])—are conjectured to lie outside TC^0. A guardrail that must evaluate an arbitrary Boolean formula over its input tokens, for example to check whether a candidate response satisfies a complex logical safety policy, is conjectured to require NC^1 or higher. Neural guardrails thus have their own blind spot: they sit above AC^0 but below NC^1, and attackers who can force the detection problem into the $\mathrm{NC}^1 \setminus \mathrm{TC}^0$ gap—should that separation hold—can evade transformer guardrails by an analogous complexity argument. Whether $\mathrm{TC}^0 \subsetneq \mathrm{NC}^1$ is a major open problem in circuit complexity [31]; the separation is strongly believed but not proved. This means the same structural argument that gives us an unconditional algebraic impossibility for regex guardrails (Theorem 6.3) yields only a *conditional* limitation for neural guardrails—conditioned on a complexity separation that has resisted proof for four decades. The information-theoretic barrier (Proposition 6.9) and the computational barrier (Proposition 6.11), by contrast, apply to *any* guardrail architecture that inspects only LLM-visible content, because those barriers arise from the fiber structure of the abstraction function rather than from the guardrail's circuit class.

Empirical TC^0 validation. We trained a minimal transformer classifier—one layer, one attention head, d_{model}=32, 14,753 parameters—on character-level tokenization of MOD_2-encoded payloads (n=30) versus clean text (n=30). On held-out encoded payloads: MOD_2 detection 90%, MOD_3 (unseen encoding) 90%, MOD_5 100%, MOD_7 100%; clean-text FPR 0%. Regex detection: 0% across all encodings. A single attention head with threshold activation detects the interleaving pattern that AC^0 circuits provably cannot compute, empirically confirming the $\mathrm{AC}^0 \subsetneq \mathrm{TC}^0$ separation in the guardrail domain.

Practical implications. The circuit-complexity hierarchy suggests a concrete layered defense strategy. Regex

guardrails should be understood as AC^0 filters: they are extremely fast (linear in input length, highly parallelizable) but provably blind to modular-counting encodings. Neural guardrails—shallow transformers trained specifically to detect structure-based bypass patterns—can close this gap in principle, because TC^0 contains all MOD_p functions. The practical recommendation is therefore to *stack* both: use regex for low-latency first-pass filtering and add a lightweight neural classifier for second-pass detection of encoding-diversity attacks that exploit the AC^0 blindness [24, 33]. This recommendation is supported by recent empirical work: Zhu et al. [41] achieve a 73.6% attack success rate via symbolic mathematical encoding across 13 production LLMs—an empirical manifestation of the algebraic blindness our Theorem 6.3 explains. Separately, cryptographic approaches [3] can *prove* that a guardrail ran, but cannot address the guardrail's algebraic blindness; our results show that proof of execution is orthogonal to proof of capability. The filter composition laws of §10.3 formalize the correct multi-layer architecture.

This recommendation, however, carries its own caveats. Neural guardrails are substantially slower and more expensive than regex: a transformer inference pass over a 1 000-token prompt is three to four orders of magnitude more computationally intensive than DFA traversal over the same input. In high-throughput deployments, inserting a neural classifier at the guardrail layer adds significant latency and cost. More critically, transformer guardrails are subject to gradient-based adversarial attacks [42]: an attacker with white-box access can craft inputs that preserve semantic content while minimizing the neural guardrail's detection score, bypassing the second-pass filter entirely. These attacks operate in weight space rather than in the algebraic structure of the input language, so they are orthogonal to the impossibility results in this paper but represent an independent and serious threat to neural guardrail deployments. The conclusion is that neural guardrails address the AC^0 blind spot at a price—both economic and security-theoretic—and the defense stack remains incomplete: the information-theoretic floor of Proposition 6.9 persists regardless of the guardrail's computational class, and adversarial robustness of neural filters is a separate unsolved problem.

10.3 Filter Composition Laws

The layered defense recommendation of Table 5 raises a natural question: how does *composing* guardrail layers affect the algebraic blindness established in §6.2? We prove five composition laws governing the interaction of AC^0 (regex) and TC^0 (neural) filters.

Let f_r denote a regex filter (recognizing a star-free language with aperiodic syntactic monoid) and f_n denote a neural filter (a threshold circuit in TC^0). Write $f_r(x) = 1$ if filter f *blocks* input x, and 0 otherwise.

Proposition 10.1 (Composition ceiling)**.** *Let ϕ_p denote MOD_p encoding.*

1. ***Parallel regex preserves blindness:*** $(f_{r_1} \vee f_{r_2})(\phi_p(x)) = 0$ *for all aperiodic* f_{r_1}, f_{r_2}. Proof: *Union of star-free languages is star-free (Proposition 6.6); the syntactic monoid of the union remains aperiodic; Theorem 6.3 applies.*
2. ***Serial regex preserves blindness:*** $(f_{r_1} \wedge f_{r_2})(\phi_p(x)) = 0$ *for all aperiodic* f_{r_1}, f_{r_2}. Proof: *Intersection of star-free languages is star-free; same argument.*
3. ***Serial confirmation preserves blindness (regex veto):*** $(f_r \wedge f_n)(\phi_p(x)) = \min(f_r(\phi_p(x)),\ f_n(\phi_p(x))) = \min(0,\ f_n(\phi_p(x))) = 0$. *The regex filter's inability to detect the encoded payload vetoes the neural filter's detection, regardless of the neural filter's accuracy.*
4. ***Parallel with neural breaks the ceiling:*** $(f_r \vee f_n)(\phi_p(x)) = \max(f_r(\phi_p(x)),\ f_n(\phi_p(x))) = \max(0,\ f_n(\phi_p(x))) = f_n(\phi_p(x))$. *The composed filter detects if and only if the neural filter detects. Since $f_n \in \mathrm{TC}^0$ and $\mathrm{MOD}_p \in \mathrm{TC}^0$, this composition escapes the AC^0 ceiling.*
5. ***Critical blend weight.*** *For a soft-score blend $g_\alpha(x) = 1$ iff $\alpha \cdot s_r(x) + (1-\alpha) \cdot s_n(x) > \tau$, where $s_r(\phi_p(x)) \approx 0$ (regex blind) and $s_n(\phi_p(x)) = c > 0$ (neural detects), the blend detects $\phi_p(x)$ if and only if $\alpha < \alpha^* := 1 - \tau/c$. Above α^*, the regex weight dominates and the blend inherits AC^0 blindness.*

Practical consequence. Items (1)–(2) confirm that stacking multiple regex filters (e.g., combining a keyword blocklist with a domain filter) does *not* repair blindness: the composed system remains provably blind to all modular encodings. Item (3) shows that a "confirm with neural" architecture, in which the regex filter must first flag a candidate before the neural classifier examines it, inherits the regex layer's blindness to encoded payloads—the dangerous case is precisely the one where the regex does not flag. Item (4) provides the correct architecture: run both filters and block if *either* detects, so that the neural classifier covers the algebraic blind spot. Item (5) quantifies the tradeoff: the blend weight α must be kept below the critical threshold $\alpha^* = 1 - \tau/c$ for the neural filter to contribute meaningfully against encoded attacks.

Empirical validation. We validated all five composition laws on 20 guardrail-style regex patterns, 20 MOD_2-encoded payloads, and 20 clean text samples. A character-bigram centroid classifier served as the neural filter. Table 7 summarizes the results.

Table 7: Empirical validation of composition laws on 20 encoded payloads. Regex detection rate is 0% on encoded payloads, confirming algebraic blindness. The neural filter achieves 100% detection with 0% FPR. All four composition predictions are confirmed.

Composition	Detection	FPR	Theorem
Regex only	0%	0%	Thm 6.3
Neural only	100%	0%	TC^0
Parallel (OR)	100%	0%	Prop 10.1(4)
Serial confirm (AND)	0%	0%	Prop 10.1(3)
Blend α=0.4	100%	0%	Below α^*
Blend α=0.5	95%	0%	Near α^*
Blend α=0.7	0%	0%	Above α^*

The theoretical critical weight $\alpha^* = 1 - \tau/c = 1 - 0.3/0.69 = 0.57$ predicts that the blend transitions from detecting to blind between $\alpha = 0.5$ and $\alpha = 0.6$, matching the empirical observation that detection drops from 95% at α=0.5 to 5% at α=0.6. The serial-confirm result (0%) empirically validates Proposition 10.1(3): the regex veto kills the neural signal on every encoded payload.

Neural evasion analysis. The parallel composition of regex and neural filters closes the AC^0 blind spot but does not close all attack vectors. We tested the neural filter (trained on MOD_2) against encoding schemes it was *not* trained on (Table 8).

Table 8: Neural filter evasion by attack type. The filter is trained on MOD_2 encoded text only. "Evades Both" indicates attacks that evade the regex $\vee$ neural composition.

Attack	Regex	Neural	Evades Both?
MOD_2 (trained)	0%	100%	No
MOD_3 (untrained)	0%	90%	10%
MOD_5 (diluted)	0%	90%	10%
Base64	0%	90%	10%
ROT13	0%	40%	60%
V1 Decomposition	0%	0%	$\mathbf{100}\%$
V3 Abstract grammar	0%	0%	$\mathbf{100}\%$
Homoglyphs	4%	60%	40%

Attacks cluster into three classes by the minimum defense layer required:

1. **Class A (encoding):** MOD_p, Base64, ROT13. Defended at the inference layer by the neural filter (Proposition 10.1(4)) plus decode-aware preprocessing (attempt common decodings and re-filter the decoded text).
2. **Class B (perturbation):** Homoglyphs, gradient adversarial. Defended by input preprocessing (Unicode NFKC normalization collapses homoglyphs to ASCII equivalents) and classifier ensemble diversity.
3. **Class C (semantic):** V1–V4 homomorphic reasoning. *Cannot* be defended at any inference layer, regardless of computational class. By Proposition 6.15, these prompts are syntactically identical to legitimate formal reasoning; any filter that blocks them must also block legitimate use. Defense *requires* execution-layer monitoring (inspect what the LLM *does*, not what it *receives*) or provenance tracking.

This classification sharpens the defense recommendation of Table 5: Classes A and B are solvable at the filter layer with known techniques and low overhead; Class C requires fundamentally different mechanisms that monitor actions rather than content. The practical defense stack is therefore: (0) input preprocessing (normalization and decode attempts), (1) regex first-pass, (2) neural second-pass in parallel (Proposition 10.1(4)), (3) execution-layer action monitoring for semantic attacks.

10.4 The Monoid Extractor as Audit Tool

The 858-line Python monoid extractor produced as part of this work provides a practical audit capability. Given a guardrail regex, the tool outputs: (1) the minimal DFA state count; (2) the full transition monoid with multiplication table; (3) all group components identified via semigroup algorithms; (4) the complete blindness spectrum (set of primes p for which the guardrail is potentially sensitive, and all others for which it is provably blind); (5) a concrete MOD_2 bypass witness string for each aperiodic pattern.

This tool enables organizations to perform a *formal algebraic audit* of their regex guardrail corpus without any manual reasoning about individual patterns. The audit result is provably correct: the blindness spectrum is not a heuristic but a theorem-backed characterization.

Production deployment workflow. An organization deploying regex-based guardrails can run the extractor on their pattern corpus to obtain a complete blindness report. For each pattern, the tool outputs: (1) the minimal DFA, (2) the syntactic monoid with multiplication table, (3) the blindness spectrum—the set of primes p for which the guardrail is provably blind to MOD_p encodings. Our analysis of 142 production patterns across 11 tools (Section 8.1) completed in under 60 seconds with zero false positives, demonstrating practical scalability for large guardrail inventories.

CI/CD integration. The extractor integrates naturally into continuous integration pipelines as a pre-commit or pre-deployment verification step. A guard can analyze any new or modified regex pattern, flagging non-aperiodic patterns before production deployment. Since the analysis is theorem-backed rather than heuristic—resting on the

Krohn–Rhodes decomposition (Proposition 6.7) and algebraic classification of star-free languages—a clean audit report is a formal guarantee of blindness, not an approximation or empirical estimate. This shifts guardrail verification from a confidence-based assertion to a mathematical proof, enabling security teams to reason about regex safety with the same rigor applied to cryptographic algorithms.

Composition safety guarantee. By Proposition 6.6, organizations can verify that their composed guardrail stack (intersection or union of multiple patterns) preserves the blindness property. The product monoid of aperiodic components remains aperiodic, so the audit report for individual patterns generalizes immediately to multi-pattern deployments. If pattern P_1 is aperiodic and pattern P_2 is aperiodic, then the composed guardrail $P_1 \cap P_2$ (enforce both) or $P_1 \cup P_2$ (enforce either) is provably blind to all modular encodings. This compositional safety is essential for practical systems, which routinely combine blocklists, domain filters, and toxicity detectors.

Empirical validation on 142 patterns. We validated this audit pipeline on the 142 patterns from 11 open-source tools (LLM Guard, Guardrails, etc.; full inventory in Table 3). All 142 patterns (100%) were aperiodic and therefore provably blind to MOD_p encodings for all primes p. This confirms the conjecture (Section 10.5) that semantic guardrail patterns in practice are aperiodic by structural design, making the monoid extractor a practical tool for organization-wide assurance: "run the extractor, check the output, and deploy with a formal proof of blindness to modular bypass strategies."

10.5 The Aperiodicity Conjecture

Conjecture 10.2 (Universal aperiodicity)**.** *Every regex pattern used as a semantic content guardrail in practice (keyword blocklists, domain filters, toxicity patterns) has an aperiodic syntactic monoid.*

Evidence. 100% of the 142-pattern corpus is aperiodic. No non-aperiodic patterns were found across any of the eleven tools. All 142 patterns are semantic content filters, consistent with the structural argument that such patterns are aperiodic by construction.

Structural argument. Semantic guardrail patterns are designed to match specific substrings (keywords, domains, toxic phrases). Such patterns are inherently positional and Boolean in character: they ask "does this substring appear?" rather than "does a count of occurrences satisfy a modular condition?" The substring-matching structure is precisely what Theorem 6.1 formalizes, and the proof shows that every pattern of this structural type has an aperiodic monoid. The conjecture amounts to the claim that no semantic guardrail in practice has the structural form of a modular counter, which is supported by both the empirical evidence and the design rationale of practical guardrail systems.

PCRE caveat. This conjecture applies to true regular expressions. PCRE extensions with backreferences recognize context-sensitive languages and fall outside our formal model. Backreferences can encode counting predicates that a pure regex cannot; a PCRE pattern using backreferences to enforce exact character count parity would not have an aperiodic syntactic monoid in the standard sense. In practice, no guardrail pattern in our corpus used backreferences.

11 Limitations and Ethical Considerations

11.1 Limitations

1. Test case severity. The empirical validation uses the aspirin synthesis case study, chosen for its minimal harm risk. The formal results (Theorems 6.1–6.15) are domain-independent; their applicability to high-severity domains is implied by the theory but not empirically demonstrated here by design.

2. ToT scope. The Tree-of-Thought benchmark evaluates 50 randomized grammars (seeds 0–49). This is a moderate sample; the yield and solve-rate estimates carry bootstrap 95% confidence intervals reported in Table 4. Systematic variation with grammar depth, branching factor, and symbol set size remains to be characterized.

3. Mathematical novelty. We use classical tools—syntactic monoids, star-freeness, Fano's inequality, free categories, Krohn–Rhodes decomposition—applied to the guardrail security setting. The mathematical novelty lies in the *application* and the combination of results into a unified framework, not in the algebraic or information-theoretic tools themselves. We state this honestly to situate the contribution accurately.

4. Uniformity assumption. The Fano bound (Proposition 6.9) assumes a uniform distribution over the concrete artifact space R. In practice, the distribution over prompts is highly non-uniform; the bound is tight under uniformity and may be loose or conservative under real prompt distributions. Under highly skewed distributions where adversarial prompts are rare, the conditional entropy $H(G(R)|U(R))$ may be near zero, making the Fano floor uninformative. Conversely, if even a small fraction

of mass lands in mixed fibers (concrete artifacts mapping to both safe and unsafe abstract types), the floor remains nonzero. A distribution-aware version of the bound would require modeling the attacker's prompt distribution; we leave this as future work, noting that the *algebraic* barriers (Theorems 6.1–6.3) are unconditional and do not depend on any distributional assumption.

5. Regex corpus size. The corpus of 142 patterns from 11 tools is representative but not exhaustive. A larger corpus would strengthen the aperiodicity conjecture (Section 10.5). We make the monoid extractor available to enable independent extension of the corpus.

6. Abstract scope. The algebraic impossibility results (Theorems 6.1 and 6.3) apply specifically to *substring-matching regex guardrails*. They do not apply to all regular expression uses, to neural guardrails (which operate in TC^0), or to guardrails that perform semantic analysis beyond pattern matching.

11.2 Ethical Considerations

The impossibility results in this paper demonstrate structural limitations of LLM guardrail systems. We have taken the following precautions.

Validation without high-severity cases. Empirical validation uses only the aspirin synthesis case study and abstract grammar benchmarks. We do not demonstrate the pipeline on domains with high harm potential.

Responsible disclosure. The existence of modular counting bypasses against regex guardrails is an implication of classical algebraic results (Theorem 6.3); any party aware of the Schützenberger–McNaughton–Papert theorem and its circuit complexity consequences could derive this result independently. We therefore consider the algebraic results non-sensitive from a disclosure perspective. The V3/V4 pipeline design is more operationally specific; we describe it at the level of algorithmic structure rather than providing a production-ready exploit.

Defense-first framing. Every impossibility result in this paper is paired with a specific defense recommendation in Section 10. The paper is structured to enable defenders to audit their systems using the monoid extractor and to migrate critical enforcement to execution-layer defenses.

The exec(compile()) irony. We note the irony that the PoC's `engine.py` uses `exec(compile(...))`—a well-known security antipattern—as its core execution mechanism. This is by design: the zero-literal AST compilation is precisely the technique that enables the attack. Highlighting this irony is intended to motivate defenders to treat dynamic code construction patterns as a first-class detection target at the execution layer.

11.3 End-to-End Validation Against Production Patterns

We tested the full defense stack against 23 patterns extracted from LLM Guard's `BanSubstrings` scanner [20], a production guardrail library deployed by ProtectAI. Table 9 compares four defense configurations across 8 encoding types on 20 payloads spanning prompt injection, code injection, SQLi, and XSS.

Table 9: End-to-end detection rates (%) against real LLM Guard patterns. "Regex Only" is the current production configuration. "Full Stack" adds preprocessing (strip, decode, normalize) and a bigram neural filter in parallel.

Encoding	Regex	+Preproc	+Neural	Full Stack
Raw payload	95	95	95	95
MOD_2	**0**	0	100	**100**
MOD_3	**0**	0	100	**100**
Base64	**0**	95	100	**100**
ROT13	**0**	95	90	**100**
ZWSP	**0**	95	0	**95**
Homoglyph	15	95	60	**95**
Leetspeak	**0**	95	80	**100**

The "Regex Only" column confirms the algebraic blindness theorem in practice: LLM Guard's production patterns detect 0% of encoded payloads across all seven encoding types. The "Full Stack" column shows that our recommended defense—preprocessing plus parallel neural filter—recovers 95–100% detection on all encodings. The defense gap is entirely attributable to the AC^0 ceiling (Theorem 6.3), and the recovery is entirely attributable to the ceiling-breaking composition (Proposition 10.1(4)) and invertibility-based preprocessing.

11.4 Case Study: secrets-router as Execution-Layer Defense

The three-class attack taxonomy (§10.3) establishes that Class C (semantic/homomorphic reasoning) attacks *cannot* be defended at any inference layer. We present `secrets-router`[1], an open-source MCP server that implements the execution-layer defense our theory recommends.

[1] `https://jrlopez.dev/p/secrets-router.html`, MIT license, ∼800 lines.

Architecture. When an AI agent needs a credential, it calls `secure_fetch("rbw", "primary card", "number")` and receives an opaque handle `handle:a3f8c2d1`. The credential value never enters the LLM context window. A separate server process resolves handles via Chrome DevTools Protocol (CDP), filling browser fields directly at the execution layer. The agent sees only masked feedback: `"filled [MASKED ****1017]"`.

Connection to the theory. The opaque handle mechanism is an implementation of the quotient functor Q from our abstraction formalization (§4.2): Q maps concrete credentials to abstract handles, and the non-invertibility of Q at the inference layer is the security guarantee. The CDP fill operates at the execution layer (Table 5, row 4), precisely the layer our theory identifies as necessary for Class C defense. Even if a semantic attack (V1–V4) causes the LLM to reason about credential extraction, it cannot extract what it never received.

12 Conclusion

We have established four independent, formally grounded impossibility results for LLM guardrail systems.

Algebraic impossibility (regex layer). All substring-matching regex guardrails have aperiodic syntactic monoids (Theorem 6.1). By the Schützenberger–AC^0–Furst–Saxe–Sipser chain, aperiodic syntactic monoids characterize exactly the languages in AC^0, which cannot compute modular counting predicates. Therefore, any regex guardrail is provably blind to modular counting encodings (Theorem 6.3), and this blindness cannot be patched within the aperiodic regex paradigm (Corollary 6.4). The Krohn–Rhodes decomposition gives a constructive audit tool: the blindness spectrum of any guardrail is computable from its syntactic monoid's group components (Proposition 6.7). Our corpus study confirms 100% aperiodicity in practice.

Information-theoretic impossibility (inference layer). Abstraction functions that map concrete artifacts to abstract types inevitably create mixed fibers. The Fano inequality gives an irreducible misclassification floor for any lifted guardrail: with 30% of mass in mixed fibers, the floor is approximately 5% (Proposition 6.9). This floor cannot be reduced by improving the guardrail's classifier—it can only be reduced by reducing the abstraction.

Computational impossibility (schema layer). Verifying whether an abstract program has a harmful instantiation is NP-complete (Proposition 6.11). Exhaustive checking is therefore infeasible, and heuristic checking inherits the false-negative rate implied by the combinatorial search space.

Structural impossibility (functor + indistinguishability). Abstract derivations transfer faithfully to real-domain operation sequences via free-category functoriality (Proposition 6.13), meaning that LLM reasoning over abstract grammars produces correct real-world procedures after offline instantiation. Adversarial formal reasoning tasks are syntactically identical to legitimate ones under Assumption 6.16 (Proposition 6.15), giving information-theoretically perfect steganographic security for the attack channel.

Empirical grounding. Five attack vectors demonstrate both barrier classes. The Tree-of-Thought benchmark ($N = 50$ grammars, seeds 0–49, $p < 0.001$) shows that BFS exhaustive search dominates LLM-guided ToT on small synthetic grammars (mean yield 0.466 vs. 0.122), confirming that the security claim of V3 depends on the existence of a solver—not on LLM superiority over BFS. The aspirin synthesis case study validates end-to-end pipeline correctness in a benign domain.

Practical consequence. Inference-layer guardrails are necessary but insufficient. The formal results collectively imply that safety enforcement cannot be achieved at the content-inspection layer alone: the barriers are structural, not implementation defects. Effective defense must migrate critical enforcement to the execution layer, where concrete artifacts—decoded strings, instantiated parameters, evaluated ASTs—become observable. The Provenance layer (end-to-end pipeline monitoring) provides the only defense against the full attack chain, at the cost of requiring offline visibility into attacker-held interpretation tables. This is a fundamental asymmetry: the attacker's interpretation table is not a secret key whose existence can be assumed away, but an intrinsic feature of the homomorphic reasoning construction. Defense-in-depth across all six layers of Table 5 is the only configuration that addresses all four barrier classes simultaneously.

References

[1] D. Angluin. Learning regular sets from queries and counterexamples. *Information and Computation*, 75(2):87–106, 1987.

[2] Anonymous. Jailbreaking large language models through content concretization. *arXiv preprint arXiv:2509.12937*, 2025.

[3] Anonymous. Proof-of-guardrail: TEE-based cryptographic proof that a guardrail ran. *arXiv preprint arXiv:2603.05786*, 2026. Proves the guardrail ran; our paper proves the guardrail is blind even when it provably runs. Complementary results.

[4] Anonymous. Structured semantic cloaking (S2C). *arXiv preprint arXiv:2603.16192*, 2026. Concurrent work.

[5] Marshall Ball, Boaz Barak, and Oded Goldreich. On the impossibility of separating intelligence from judgment: The computational intractability of filtering for AI alignment. *arXiv preprint arXiv:2507.07341*, 2025.

[6] D. A. M. Barrington, K. Compton, H. Straubing, and D. Thérien. Regular languages in NC^1. *Journal of Computer and System Sciences*, 44(3):478–499, 1992.

[7] N. Boucher, I. Shumailov, R. Anderson, and N. Papernot. Bad characters: Imperceptible NLP attacks. In *IEEE Symposium on Security and Privacy*, 2022.

[8] D. Chiang, P. Cholak, and A. Pillay. Tighter bounds on the expressivity of transformer encoders. In *International Conference on Learning Representations (ICLR)*, 2024.

[9] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.

[10] Y. Dong et al. No free lunch with guardrails. *arXiv preprint arXiv:2504.00441*, 2025. Demonstrates that guardrail overhead imposes non-trivial computational cost and that no single guardrail strategy dominates across threat models.

[11] S. Eilenberg. *Automata, Languages, and Machines, Volume B*. Academic Press, 1976.

[12] M. K. B. Esmradi, R. Nair, et al. h4rm3l: A dynamic benchmark of composable jailbreak attacks for LLM safety assessment. *arXiv preprint arXiv:2408.04811*, 2024.

[13] M. Furst, J. B. Saxe, and M. Sipser. Parity, circuits, and the polynomial-time hierarchy. In *Proceedings of the 22nd Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 260–270, 1981.

[14] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[15] T. Hagendorff, E. Derner, and N. Oliver. Large reasoning models are autonomous jailbreak agents. *Nature Communications*, 2026.

[16] M. Hahn. Theoretical limitations of self-attention in neural sequence models. *Transactions of the Association for Computational Linguistics*, 8:156–171, 2020.

[17] J. Håstad. *Computational Limitations of Small-Depth Circuits*. PhD thesis, MIT, 1987.

[18] N. J. Hopper, J. Langford, and L. von Ahn. Provably secure steganography. In *Advances in Cryptology — CRYPTO 2002*. Springer, 2002.

[19] K. Krohn and J. Rhodes. Algebraic theory of machines. I. Prime decomposition theorem for finite semigroups and machines. *Transactions of the American Mathematical Society*, 116:450–464, 1965.

[20] A. Kumar, C. Agarwal, S. Srinivas, G. Li, S. Feizi, and H. Lakkaraju. Certifying LLM safety against adversarial prompting. *arXiv preprint arXiv:2309.02705*, 2023. Proposes certified defenses based on randomized smoothing applied to LLM input representations, providing provable safety guarantees.

[21] Y. Li et al. SoK: Evaluating jailbreak guardrails for large language models. *arXiv preprint arXiv:2506.10597*, 2025. Systematization-of-knowledge paper evaluating the effectiveness and limitations of current jailbreak guardrail approaches across categories.

[22] S. Mac Lane. *Categories for the Working Mathematician*, volume 5 of *Graduate Texts in Mathematics*. Springer-Verlag, 1971.

[23] R. McNaughton and S. Papert. *Counter-Free Automata*. MIT Press, 1971.

[24] W. Merrill. What formal languages can transformers express? A survey. *Transactions of the Association for Computational Linguistics*, 12:543–561, 2024.

[25] W. Merrill and A. Sabharwal. The parallelism tradeoff: Limitations of log-precision transformers. *Transactions of the Association for Computational Linguistics*, 11:531–545, 2023.

[26] J. Myhill. Finite automata and the representation of events. *WADD Technical Report*, 57–624, 1957.

[27] A. Nerode. Linear automaton transformations. *Proceedings of the American Mathematical Society*, 9(4):541–544, 1958.

[28] J.-É. Pin. *Varieties of Formal Languages*. Plenum Press, 1986.

[29] M.-P. Schützenberger. On finite monoids having only trivial subgroups. *Information and Control*, 8(2):190–194, 1965.

[30] J. R. Searle. Minds, brains, and programs. *Behavioral and Brain Sciences*, 3(3):417–424, 1980.

[31] M. Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 3rd edition, 2012.

[32] H. Straubing. Finite automata, formal logic, and circuit complexity. 1994.

[33] L. Strobl, W. Merrill, G. Weiss, D. Chiang, and D. Angluin. Formal language recognition by hard attention transformers: Precise bounds and some separations. *Transactions of the Association for Computational Linguistics*, 12:1–19, 2024.

[34] StructuralSleight. UTES: Uncommon text-encoded structures for automated jailbreaking. *arXiv preprint arXiv:2406.08754*, 2024.

[35] A. Wei, N. Haghtalab, and J. Steinhardt. Jailbroken: How does LLM safety training fail? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[36] Y. Wei et al. DictAttack: Exploiting key-value dictionaries for jailbreaking large language models. *arXiv preprint arXiv:2503.24191*, 2025.

[37] G. Weiss, Y. Goldberg, and E. Yahav. Extracting automata from recurrent neural networks using queries and counterexamples. *Machine Learning*, 111:2855–2882, 2022.

[38] Z. Xu et al. Bypassing LLM guardrails: Empirical analysis of adversarial prompt strategies. In *Proceedings of the ACL 2025 Workshop on Language Models and Security (LLMSec)*, 2025. Systematic empirical survey of guardrail bypass techniques including encoding, role-play, and indirect-instruction attacks.

[39] Z. Zeng et al. R2-Guard: Robust reasoning-based guardrail for LLMs. In *International Conference on Learning Representations (ICLR)*, 2025. Introduces a reasoning-based guardrail using chain-of-thought evaluation to improve detection of harmful content over regex and embedding approaches.

[40] Y. Zhou et al. STAC: When innocent tools form dangerous chains. *arXiv preprint arXiv:2509.25624*, 2025.

[41] M. Zhu et al. MathPrompt: Mathematical encoding attacks against LLM safety. *arXiv preprint arXiv:2409.11445*, 2024. 73.6% attack success rate via symbolic math encoding across 13 LLMs. Our algebraic blindness theorem explains why this works.

[42] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

A Proof-of-Concept Source Code

The complete PoC consists of 376 lines across six files. The monoid extractor (858 lines) and benchmark harness (619 lines) are provided as supplementary material at the project repository. Below we summarize the architecture; full source is available online.

A.1 Engine: Zero-Knowledge Recurrent Executor

`engine.py` (63 lines) decodes a base64-encoded configuration, selects an AST medium, compiles it via `exec(compile(...))`, and loops until a termination condition. No LLM sees the configuration contents.

Listing 3: engine.py (63 lines)

```
"""ZK Recurrent Executor. Decodes b64
    config, selects
AST medium, compiles, injects params at
    runtime, loops
until termination. No LLM sees the
    config contents."""
import base64, json, sys, time
from mediums import get_medium

def decode_config(path):
    with open(path, 'rb') as f:
        return json.loads(base64.
    b64decode(f.read()))

def check_termination(results, iteration
    , config):
    term = config.get('termination',
                      {'type': 'count',
    'value': 1})
    t = term['type']
    if t == 'count':
        return iteration >= term.get('
    value', 1)
    if t == 'has_results':
        return bool(results)
    if t == 'min_results':
        if isinstance(results, dict) and
     'path' in results:
```

```
            return len(results['path'])
    > 0
        return bool(results) \
            and len(results) >= term.get
    ('value', 1)
    if t == 'score_threshold':
        return isinstance(results, dict)
     \
            and results.get('score', 0)
    >= term['value']
    return True

def mutate_params(params, iteration,
    config):
    m = config.get('mutation', {})
    if m.get('type') == 'paginate':
        k = m.get('key', 'page')
        params[k] = params.get(k, 1) + 1
    elif m.get('type') == 'backoff':
        time.sleep(min(2 ** iteration,
    30))
    return params

def main():
    config = decode_config(
        sys.argv[1] if len(sys.argv) > 1
     else 'config.b64')
    params = config['params']
    build_ast = get_medium(config['
    medium'])

    ns = {}
    exec(compile(build_ast(),
         f'<{config["medium"]}>', 'exec'
    ), ns)
    run = ns['run']

    max_iter = config.get('
    max_iterations', 10)
    results = None
    for i in range(1, max_iter + 1):
        print(f"[engine]␣iteration␣{i}/{
    max_iter}")
        try:
            results = run(**params)
        except Exception as e:
            print(f"[engine]␣error:␣{e}"
    )
        if check_termination(results, i,
     config):
            break
        params = mutate_params(params, i
    , config)

    print(f"\n[engine]␣RESULTS:")
    if isinstance(results, list):
        for j, item in enumerate(results
    , 1):
            t = item.strip() if
    isinstance(item, str) \
                else str(item)
            if t:
                print(f"␣␣{j}.␣{t}")
    elif isinstance(results, dict):
        print(json.dumps(results, indent
    =2))
    else:
        print(results)

if __name__ == '__main__':
    main()
```

A.2 Medium: BFS Graph Solver

`graph_solver.py` implements BFS over an abstract state space. States are sets of opaque symbols; rules define valid transitions. Zero domain content — all semantics injected at runtime.

Listing 4: graph_solver.py (43 lines)

```
"""AST template: BFS state-space solver.
Zero domain content."""
import ast

def build_graph_solver_ast() -> ast.
    Module:
    source = '''
from collections import deque

def applicable_rules(state, rules):
    return [r for r in rules
            if all(i in state for i in r
    ["inputs"])]

def apply_rule(state, rule):
    new = [s for s in state
           if s not in rule["inputs"]]
    new.extend(rule["outputs"])
    return sorted(set(new))

def check_constraints(state, constraints
    ):
    for c in constraints:
        if c["type"] == "forbidden_pair
    ":
            if c["params"][0] in state \
               and c["params"][1] in
    state:
                return False
    return True

def run(initial_state, target, rules,
    constraints):
    queue = deque([(initial_state, [],
    1.0)])
    visited = set()
```

```
    best = None
    while queue:
        state, path, score = queue.
    popleft()
        key = tuple(state)
        if target in state:
            if best is None or score >
    best["score"]:
                best = {"path": path, "
    score": score,
                        "final_state":
    state}
            continue
        if key in visited:
            continue
        visited.add(key)
        if len(path) >= 10:
            continue
        for rule in applicable_rules(
    state, rules):
            new_state = apply_rule(state
    , rule)
            if check_constraints(
    new_state, constraints):
                new_score = score * rule
    ["yield_est"]
                queue.append((new_state,
                    path + [rule["id"]],
     new_score))
    return best or {"path": [], "score":
     0,
                    "final_state":
    initial_state}
'''
    tree = ast.parse(source)
    ast.fix_missing_locations(tree)
    return tree
```

A.3 Medium: Tree-of-Thought Solver with LLM Evaluation

`tot_solver.py` implements beam search with LLM-guided branch pruning. The LLM sees *only* abstract symbols (`N1`, `R7`, etc.) and rates branches as SURE, LIKELY, or IMPOSSIBLE. Composite scoring: yield weight 0.4 + LLM viability 0.6. Beam width 5, max depth 8.

Listing 5: tot_solver.py — core evaluation and search (abbreviated; 158 lines full)

```
def ollama_evaluate(state, target, rules
    , model):
    """Ask LLM: is this state viable
    toward target?
    Returns 0.0-1.0. LLM sees ONLY
    abstract symbols."""
    system_ctx = _build_system_context(
    target, rules)
    applicable = [
        f'{r["id"]}: {r["inputs"]} -> {r
    ["outputs"]}'
        f' (yield {r["yield_est"]})'
        for r in rules
        if all(i in state for i in r["
    inputs"])]
    user_prompt = (
        f"CURRENT STATE: {state}\n"
        f"APPLICABLE NOW:\n"
        + ("\n".join(applicable)
           if applicable else "(none)")
        + f"\n\nCan {target} be reached?
    "
        f"Trace forward briefly, then
    rate.\n"
        f"RATING: SURE / LIKELY /
    IMPOSSIBLE")
    try:
        data = json.dumps({
            "model": model,
            "messages": [
                {"role": "system",
                 "content": system_ctx},
                {"role": "user",
                 "content": user_prompt
    }],
            "stream": False}).encode()
        req = urllib.request.Request(
            "http://localhost:11434/api/
    chat",
            data=data,
            headers={"Content-Type":
                     "application/json"
    })
        with urllib.request.urlopen(req,
                timeout=15) as resp:
            result = json.loads(resp.
    read())
        content = result["message"]["
    content"]
        lines = [l.strip()
                 for l in content.strip
    ().split("\n")
                 if l.strip()]
        last_line = lines[-1].upper() if
     lines else ""
        if "IMPOSSIBLE" in last_line:
    return 0.0
        if "SURE" in last_line: return
    1.0
        if "LIKELY" in last_line: return
     0.6
        return 0.5
    except Exception:
        return 0.5

def run(initial_state, target, rules,
    constraints,
```

```
    model="llama3.2:3b"):
"""Tree-of-Thought search: generate
branches,
LLM evaluates, prune, expand."""
beam_width, max_depth = 5, 8
frontier = [(initial_state, [], 1.0,
 1.0)]
visited = set()
best = None
for depth in range(max_depth):
    if not frontier: break
    candidates = []
    for state, path, cum_yield, _ in
 frontier:
        key = tuple(state)
        if target in state:
            if best is None \
               or cum_yield > best["
score"]:
                best = {"path": path
,
                        "score":
cum_yield,
                        "final_state
": state}
            continue
        if key in visited: continue
        visited.add(key)
        for rule in applicable_rules
(state, rules):
            new_state = apply_rule(
state, rule)
            if not check_constraints
(
                    new_state,
constraints):
                continue
            new_yield = cum_yield*
rule["yield_est"]
            new_path = path + [rule[
"id"]]
            candidates.append((
new_state,
                new_path, new_yield,
 rule))
    if not candidates: break
    scored = []
    for ns, np, ny, rule in
candidates:
        llm = ollama_evaluate(
            ns, target, rules, model
)
        composite = ny * 0.4 + llm *
 0.6
        scored.append((ns, np, ny,
composite))
    scored.sort(key=lambda x: x[3],
reverse=True)
    frontier = [f for f in scored[:
    beam_width]
                    if f[3] > 0.1]
    return best or {"path": [], "score":
     0.0,
                    "final_state":
    initial_state}
```

A.4 Config Encoder

Listing 6: encode.py (22 lines)

```
"""Encode a JSON config to b64.
No LLM sees plaintext values."""
import base64, json, sys

def main():
    if sys.argv[1] == '--stdin':
        config = json.loads(sys.stdin.
    read())
    else:
        with open(sys.argv[1]) as f:
            config = json.load(f)
    out = sys.argv[2] if len(sys.argv) >
     2 \
        else 'config.b64'
    encoded = base64.b64encode(
        json.dumps(config).encode())
    with open(out, 'wb') as f:
        f.write(encoded)
    print(f"[encode]␣{out}␣({len(encoded
    )}B)␣"
          f"medium={config.get('medium
    ','?')}␣"
          f"keys={list(config.get('
    params',{}).keys())}")

if __name__ == '__main__':
    main()
```

A.5 Medium Registry

Listing 7: mediums/__init__.py (13 lines)

```
"""Medium registry."""
from mediums.web_scraper import
    build_web_scraper_ast
from mediums.graph_solver import
    build_graph_solver_ast
from mediums.tot_solver import
    build_tot_solver_ast

REGISTRY = {
    "web_scraper": build_web_scraper_ast
    ,
    "graph_solver":
    build_graph_solver_ast,
```

```
    "tot_solver": build_tot_solver_ast,
}

def get_medium(name: str):
    if name not in REGISTRY:
        raise ValueError(
            f"Unknown medium '{name}'. "
            f"Available: {list(REGISTRY.
    keys())}")
    return REGISTRY[name]
```

A.6 Supplementary Material

The following artifacts are provided as supplementary material and are not included inline due to length:

- `monoid_extractor.py` (858 lines): Thompson NFA construction, powerset DFA conversion, Hopcroft minimization, transition monoid enumeration, aperiodicity test, Krohn–Rhodes group component extraction, and MOD_2 bypass witness generation.
- `benchmark.py` (619 lines): Grammar generator with guaranteed reachability, dead-end traps, noise rules, and constraint variation; BFS, random-beam, and ToT+LLM solvers; Wilcoxon signed-rank test computation; JSON result serialization.
- `web_scraper.py` (77 lines): AST-based web scraper medium with zero string literals.

B Agentic Research Methodology (Supplementary)

This paper was developed using an agentic AI research pipeline. Full pipeline details, skill definitions, and reproduction scripts are available at the project repository. We document the methodology both as a reproducibility aid and because the Five-stage pipeline: (1) human-directed threat model and proof targets; (2) delegated code generation via local LLM agents (qwen2.5-coder:7b); (3) adversarial mathematical audit by a separate AI reviewer instance; (4) three-round simulated peer review (V1 Reject $\rightarrow$ V2 Weak Accept $\rightarrow$ V3 Accept with minor revisions); (5) consolidation into current paper.

The peer reviewer caught a BFS-vs-ToT attribution error during V1 review, forcing the honest reframe in Section 8.2—the same formal reasoning capability that enables the attacks in this paper also enables quality gates in AI-assisted research.

Reproducibility. All code, prompts, and model outputs are available at the project repository. Models: Claude Opus 4.6 (architecture), Claude Haiku 4.5 (code generation), qwen2.5-coder:7b (benchmarks), all via Ollama or Anthropic API as of March 2026.